CSE113: Parallel Programming

- **Topics**:
  - Scheduling

Announcements

- HW#2 graded
- Midterm being graded. By Thursday.

- Tips to improve understanding going into the final:
  - Attend class, study the slides
  - Do readings
  - Experiment with homework
  - Focus on understanding and problem solving

Announcements

- HW3 due on Thursday (+ 3 days if you need it)

Announcements

- Module 4: GPU for three sessions

- Module 5: advanced topics – Concurrent sets, barriers and memory models

Schedule

- DOALL Loops

- **Parallel Schedules:**
  - Static
  - **Global Worklists**
  - Local Worklists

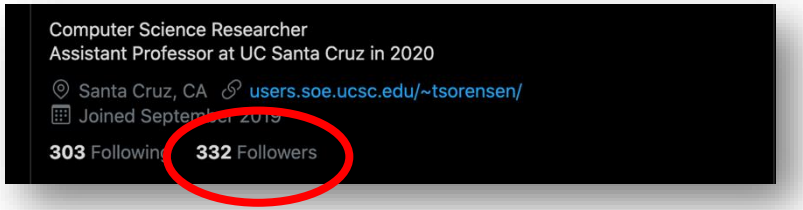
Irregular parallelism in loops

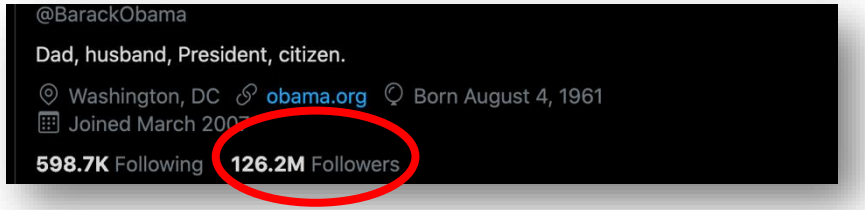
- Tasks are not balanced
- Appears in lots of emerging workloads

Irregular parallelism in loops

- Tasks are not balanced
- Appears in lots of emerging workloads

  social network analytics where threads are parallel across users





Irregular parallelism in loops

- Independent iterations have different amount of work to compute
- Threads with longer tasks take longer to compute.
- Threads with shorter tasks are under utilized.

```
for (x = 0; x < SIZE; x++) {
  for (y = x; y < SIZE; y++) {
    a[x,y] = b[x,y] + c[x,y];
  }
}
```

irregular (or unbalanced) parallelism: each x iteration performs different amount of work.

Irregular parallelism in loops

- Calculate imbalance cost if x is chunked:
  - Thread 1 takes iterations 0 - SIZE/2
  - Thread 2 takes iterations SIZE/2 - SIZE

```
for (x = 0; x < SIZE; x++) {
  for (y = x; y < SIZE; y++) {
    a[x,y] = b[x,y] + c[x,y];
  }
}
```

Irregular parallelism in loops

- Calculate imbalance cost if x is chunked:
  - Thread 1 takes iterations 0 - SIZE/2
  - Thread 2 takes iterations SIZE/2 - SIZE

Calculate how much total work:

$$\text{total_work} = \sum_{n=0}^{SIZE} n$$

```
for (x = 0; x < SIZE; x++) {
  for (y = x; y < SIZE; y++) {
    a[x,y] = b[x,y] + c[x,y];
  }
}
```

Irregular parallelism in loops

- Calculate imbalance cost if x is chunked:
  - Thread 1 takes iterations 0 - SIZE/2
  - Thread 2 takes iterations SIZE/2 - SIZE

```
for (x = 0; x < SIZE; x++) {
  for (y = x; y < SIZE; y++) {
    a[x,y] = b[x,y] + c[x,y];
  }
}
```

Calculate how much total work:

$$\text{total_work} = \sum_{n=0}^{SIZE} n$$

Calculate work done by second thread:

$$\text{t2_work} = \sum_{n=0}^{SIZE/2} n$$

Irregular parallelism in loops

- Calculate imbalance cost if x is chunked:
  - Thread 1 takes iterations 0 - SIZE/2
  - Thread 2 takes iterations SIZE/2 - SIZE

```
for (x = 0; x < SIZE; x++) {
  for (y = x; y < SIZE; y++) {
    a[x,y] = b[x,y] + c[x,y];
  }
}
```

Calculate how much total work:

$$\text{total_work} = \sum_{n=0}^{SIZE} n$$

Calculate work done by second thread:

$$\text{t2_work} = \sum_{n=0}^{SIZE/2} n$$

Calculate work work done by first thread:

t1_work = total_work − t2_work

Irregular parallelism in loops

Example: SIZE = 64

total_work = 2016
t2_work = 496
t1_work = 1520

t1 does ~3x more work than t2

Only provides ~1.3x speedup

Potential solution:
Have T1 do only ¼ of the iterations
Gives a better speedup of 1.77x

Not a feasible solution because often times load imbalance is not given by a static equation on loop bounds!

Calculate how much total work:

$$\text{total_work} = \sum_{n=0}^{SIZE} n$$

Calculate work done by first thread:

$$\text{t1_work} = \sum_{n=0}^{SIZE/2} n$$

Calculate work work done by second thread:

t2_work = total_work − t1_work

Work stealing

- Tasks are dynamically assigned to threads.

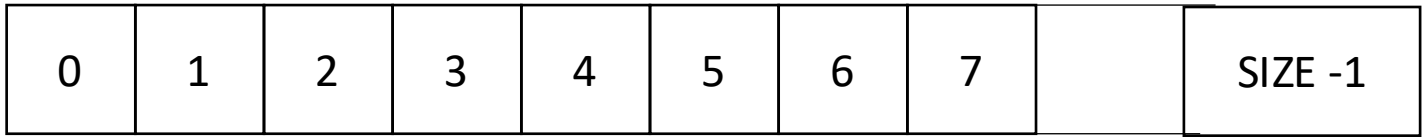
Work stealing - global worklist

- Pros
  - Simple to implement

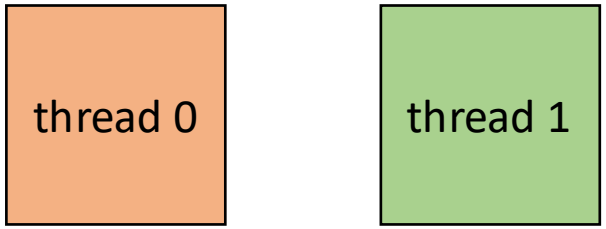
- Cons:
  - High contention on global counter
  - Potentially bad memory locality.

Work stealing - global worklist

- Global worklist: threads take tasks (iterations) dynamically

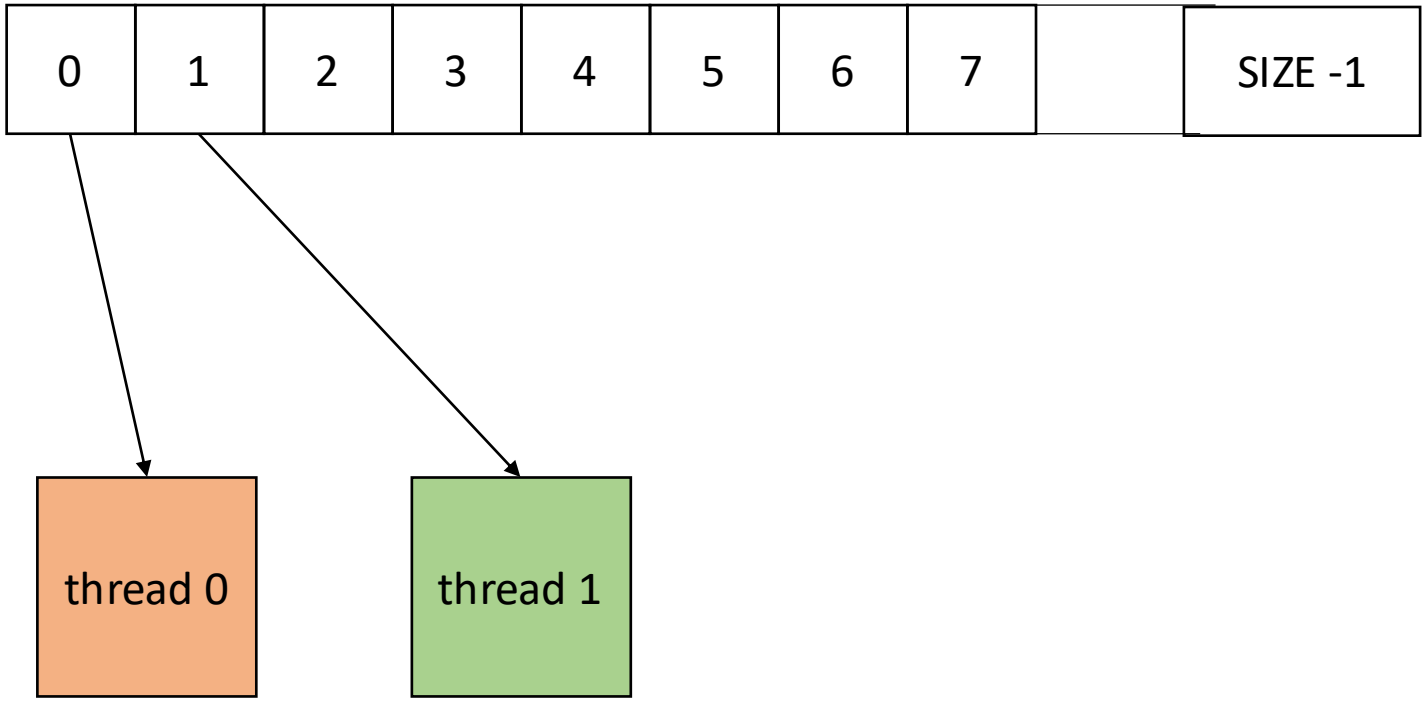


cannot color initially!



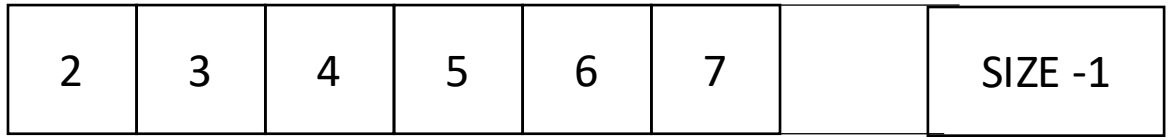
Work stealing - global worklist

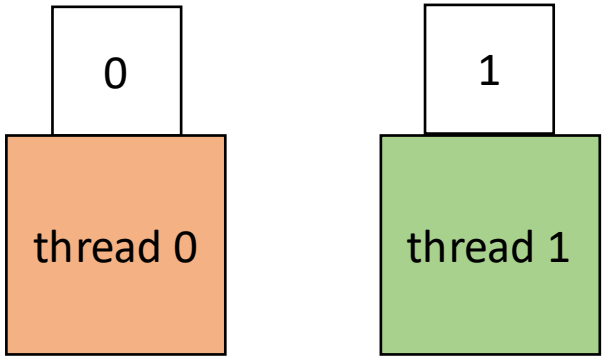
- Global worklist: threads take tasks (iterations) dynamically



Work stealing - global worklist

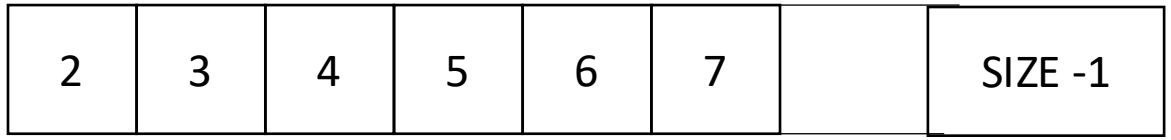
- Global worklist: threads take tasks (iterations) dynamically

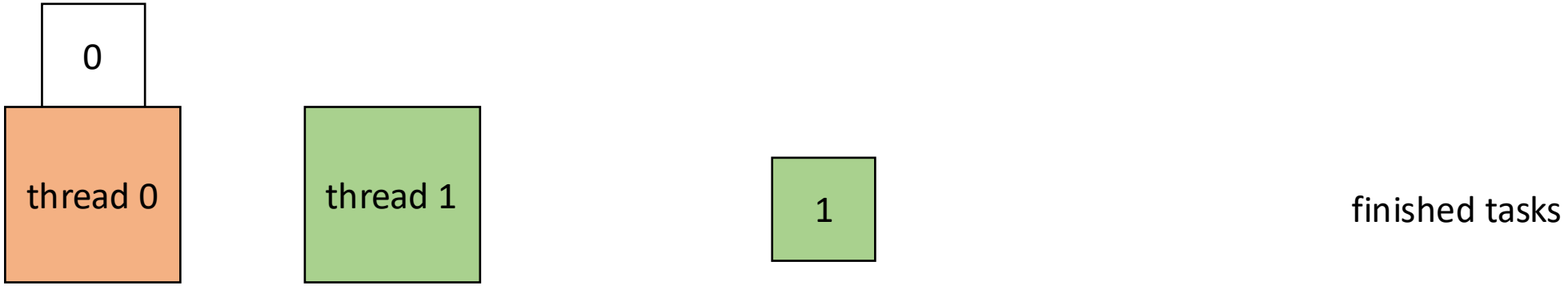




Work stealing - global worklist

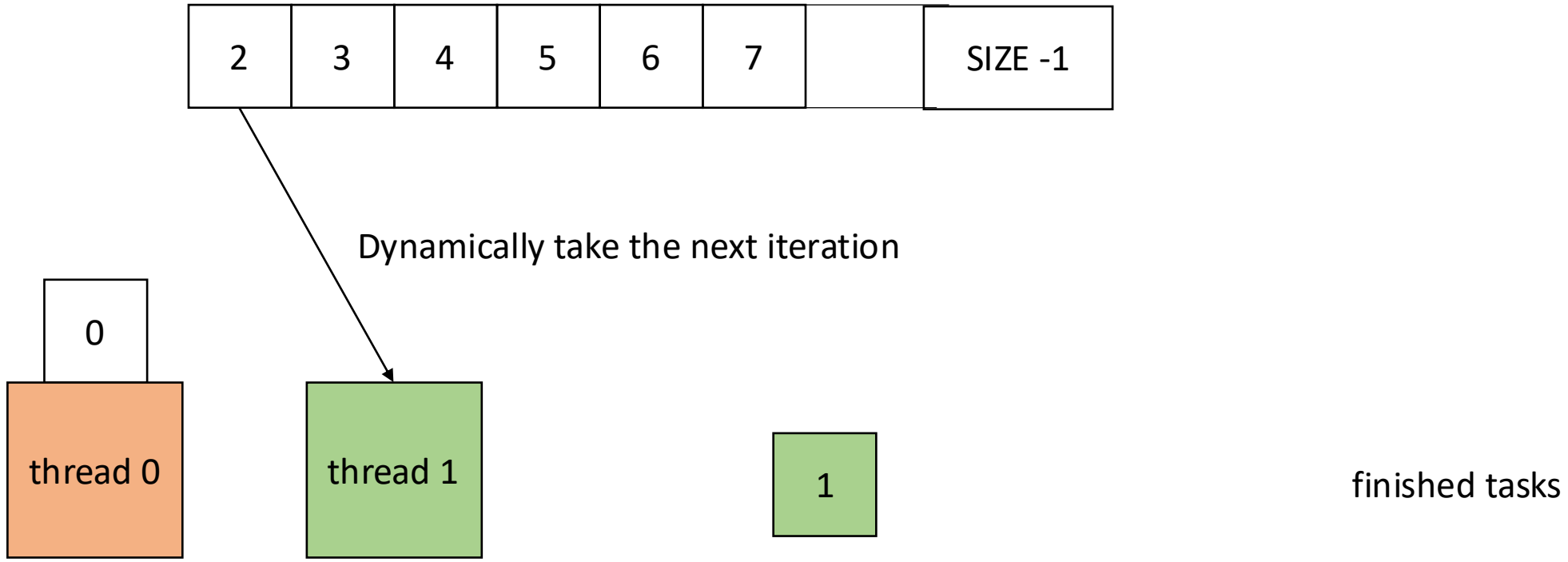
- Global worklist: threads take tasks (iterations) dynamically





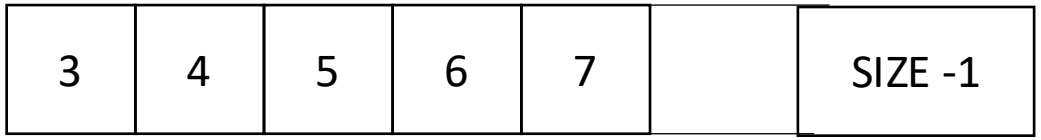
Work stealing - global worklist

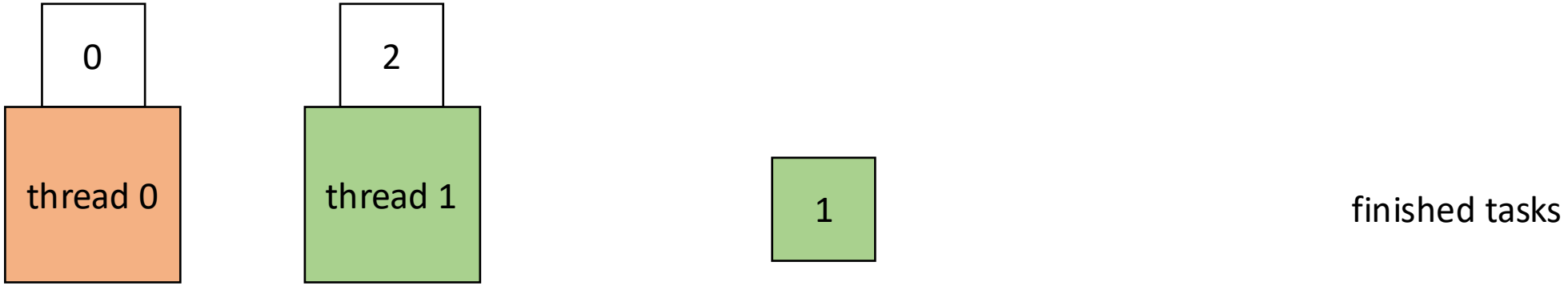
- Global worklist: threads take tasks (iterations) dynamically



Work stealing - global worklist

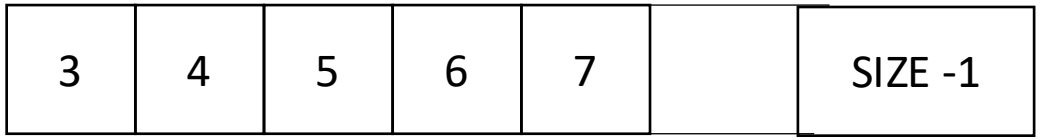
- Global worklist: threads take tasks (iterations) dynamically

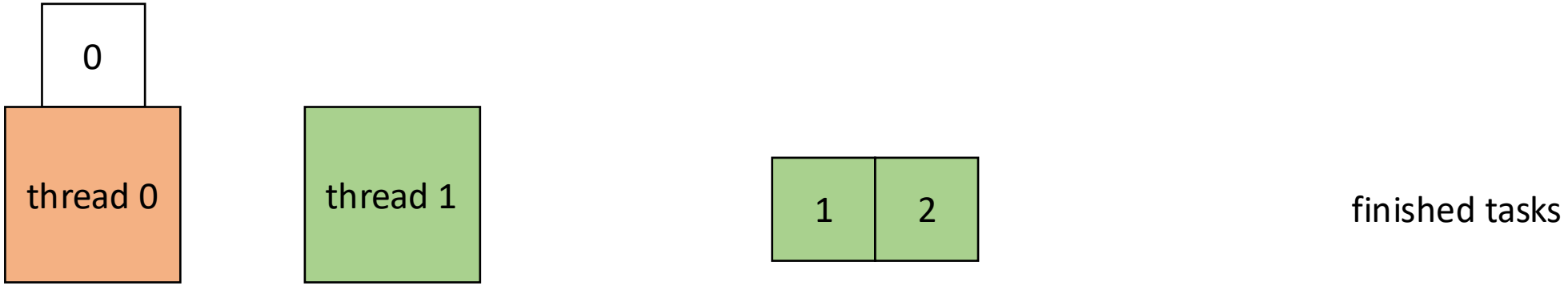




Work stealing - global worklist

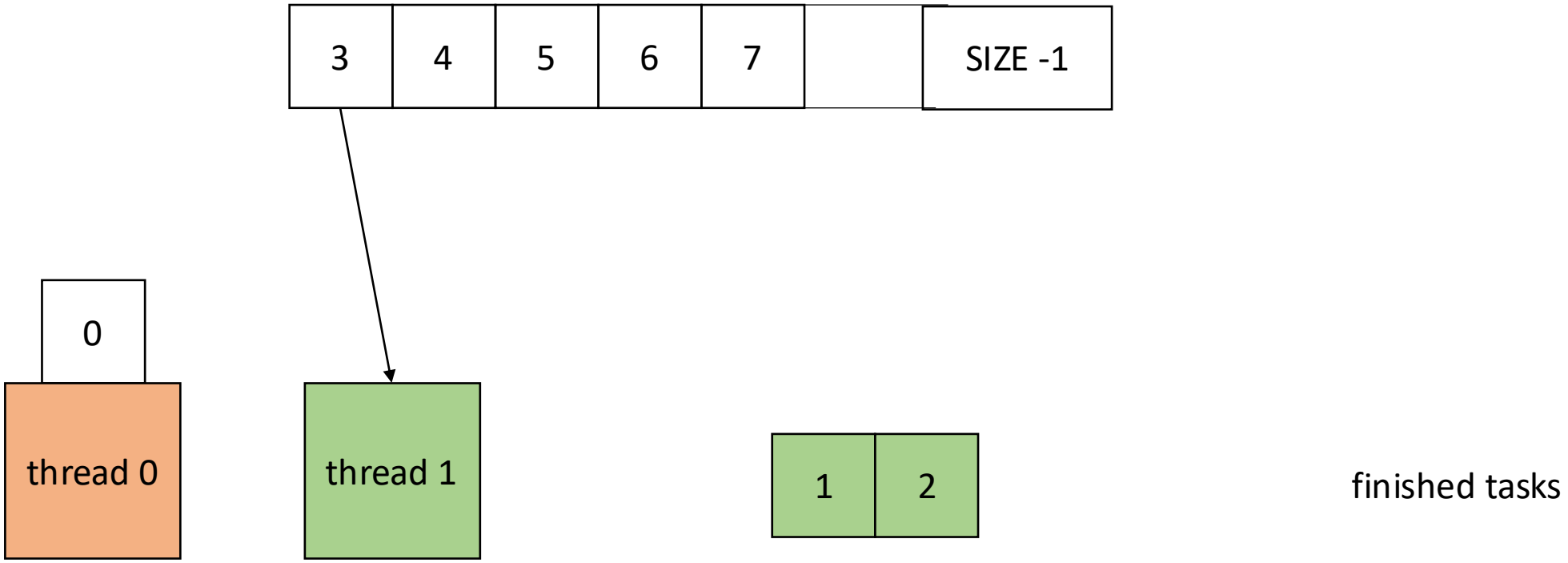
- Global worklist: threads take tasks (iterations) dynamically





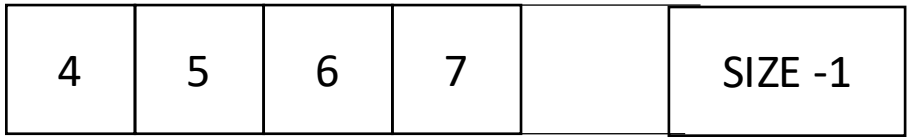
Work stealing - global worklist

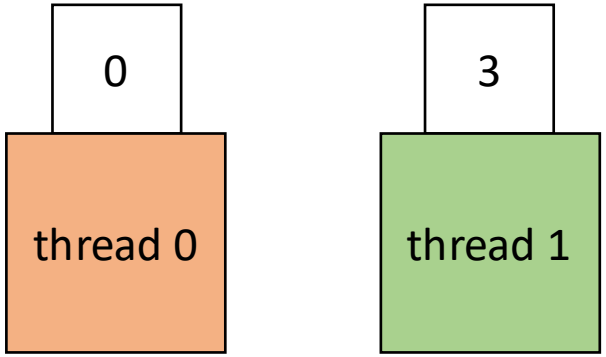
- Global worklist: threads take tasks (iterations) dynamically

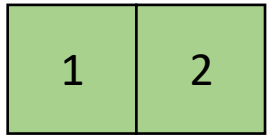


Work stealing - global worklist

- Global worklist: threads take tasks (iterations) dynamically



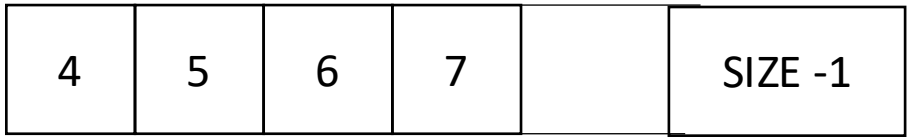


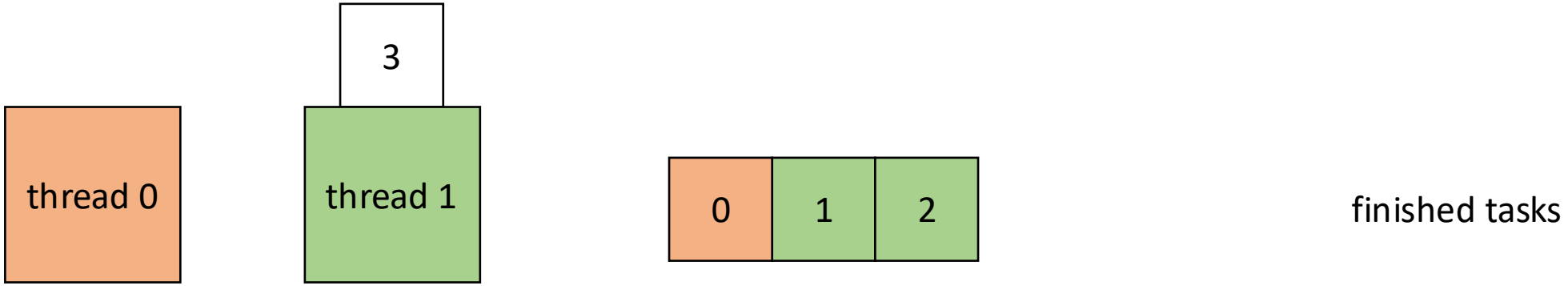


finished tasks

Work stealing - global worklist

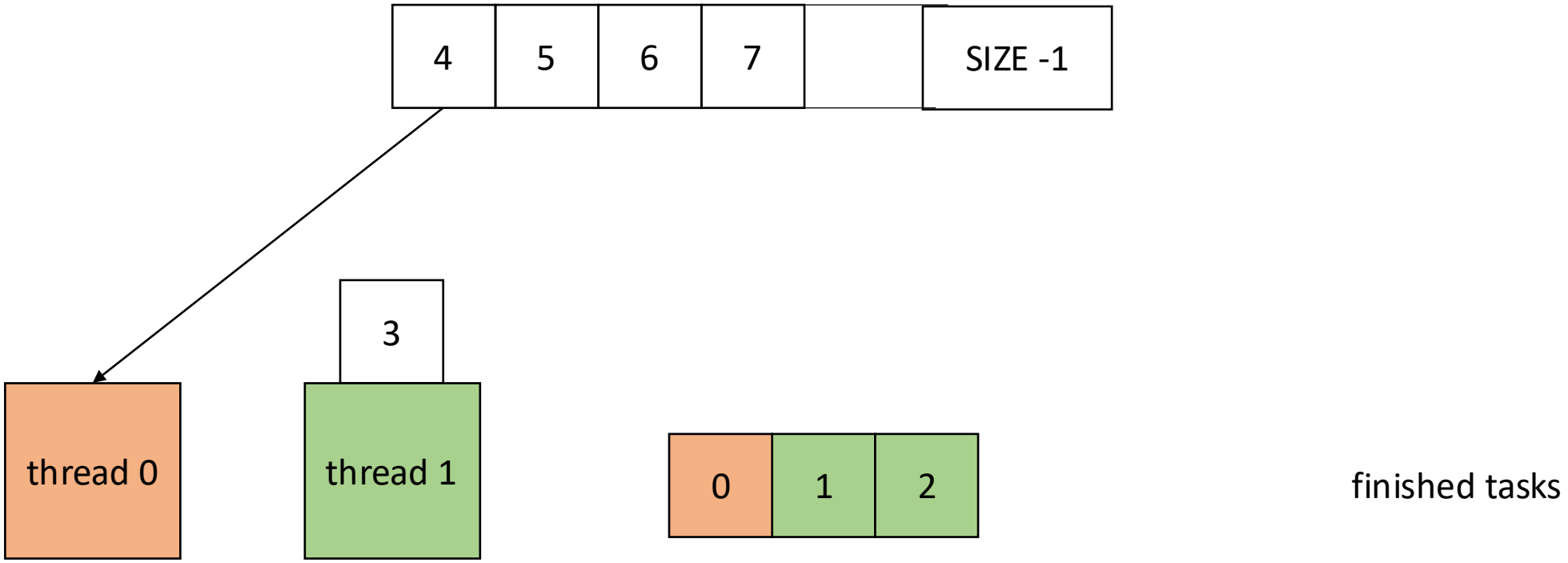
- Global worklist: threads take tasks (iterations) dynamically





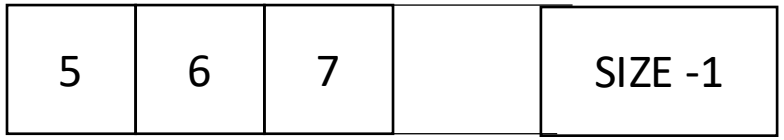
Work stealing - global worklist

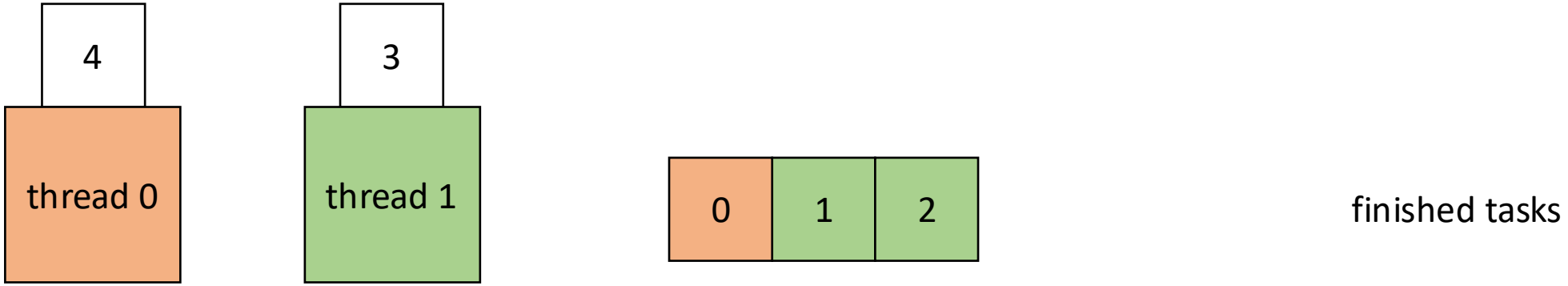
- Global worklist: threads take tasks (iterations) dynamically



Work stealing - global worklist

- Global worklist: threads take tasks (iterations) dynamically





Work stealing - global worklist

- How to implement

```
void foo() {
  ...
  for (x = 0; x < SIZE; x++) {
    // dynamic work based on x
  }
  ...
}
```

Work stealing - global worklist

- How to implement

```
void foo() {
   ...
   for (x = 0; x < SIZE; x++) {
     // dynamic work based on x
   }
   ...
}
```

```
void parallel_loop(...) {

  for (x = 0; x < SIZE; x++) {
    // dynamic work based on x
  }
}
```

Replicate code in a new function. Pass all needed variables as arguments.

Work stealing - global worklist

- How to implement

```
void foo() {
   ...
   for (x = 0; x < SIZE; x++) {
     // dynamic work based on x
   }
   ...
}
```

```
atomic_int x(0);
void parallel_loop(...) {

  for (x = 0; x < SIZE; x++) {
    // dynamic work based on x
  }
}
```

move loop variable to be a global atomic variable

Work stealing - global worklist

- How to implement

```
void foo() {
   ...
   for (x = 0; x < SIZE; x++) {
      // dynamic work based on x
   }
   ...
}
```

```
atomic_int x(0);
void parallel_loop(...) {

   for (int local_x = ??
            local_x < SIZE;
            local_x = ??) {

      // dynamic work based on x
   }
}
```

change loop bounds in new function to use a local variable using global variable.

Work stealing - global worklist

- How to implement

These must be atomic updates!

```
void foo() {
   ...
   for (x = 0; x < SIZE; x++) {
      // dynamic work based on x
   }
   ...
}
```

```
atomic_int x(0);
void parallel_loop(...) {

   for (int local_x = atomic_fetch_add(&x,1);
            local_x < SIZE;
            local_x = atomic_fetch_add(&x,1)) {

      // dynamic work based on x
   }
}
```

change loop bounds in new function to use a local variable using global variable.

Work stealing - global worklist

- How to implement

```
void foo() {
  ...
  for (t = 0; t < THREADS; t++) {
    spawn(parallel_loop);
  }
  join();
  ...
}
```

```
atomic_int x(0);
void parallel_loop(...) {

  for (int local_x = atomic_fetch_add(&x,1);
            local_x < SIZE;
            local_x = atomic_fetch_add(&x,1)) {

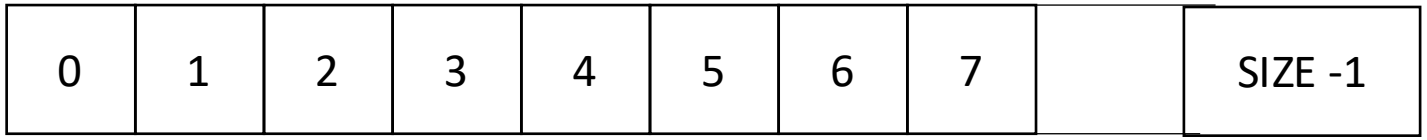
     // dynamic work based on x
  }
}
```

Spawn threads in original function and join them afterwards

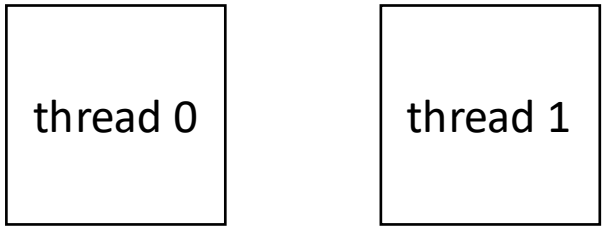
You will have to change to C++ syntax for the homework!

Work stealing - global worklist

- Global worklist: threads take tasks (iterations) dynamically




```
x: 0
0 - local_x - UNDEF
1 - local_x - UNDEF
```



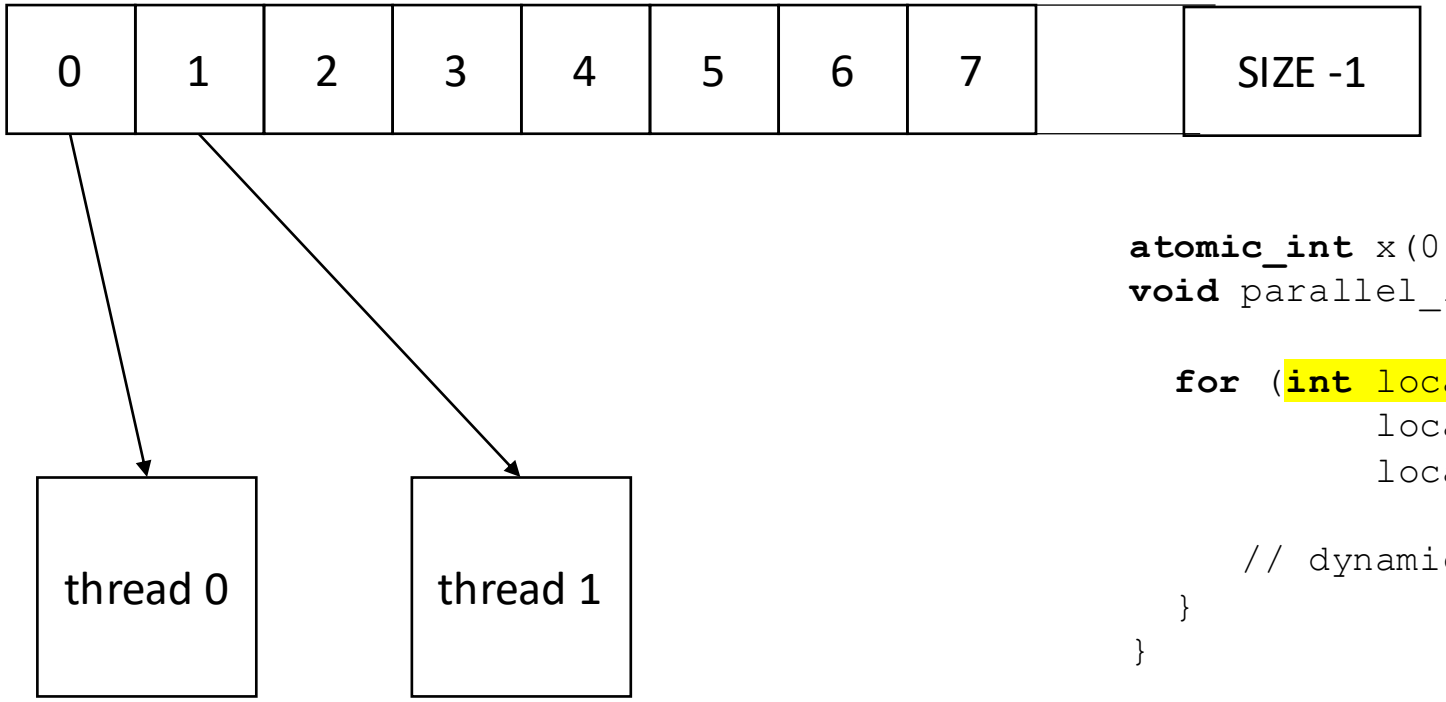

```
atomic_int x(0);
void parallel_loop(...) {

  for (int local_x = atomic_fetch_add(&x,1);
           local_x < SIZE;
           local_x = atomic_fetch_add(&x,1)) {

     // dynamic work based on x
  }
}
```

Work stealing - global worklist

- Global worklist: threads take tasks (iterations) dynamically




```
x: 2
0 - local_x - 0
1 - local_x - 1
```

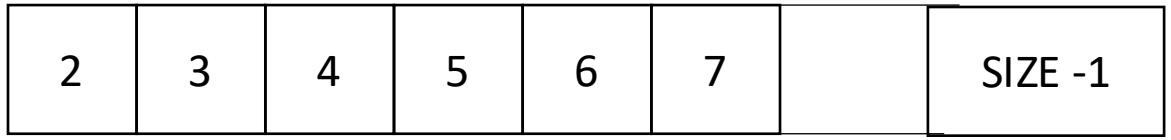
```
atomic_int x(0);
void parallel_loop(...) {

  for (int local_x = atomic_fetch_add(&x,1);
            local_x < SIZE;
            local_x = atomic_fetch_add(&x,1)) {

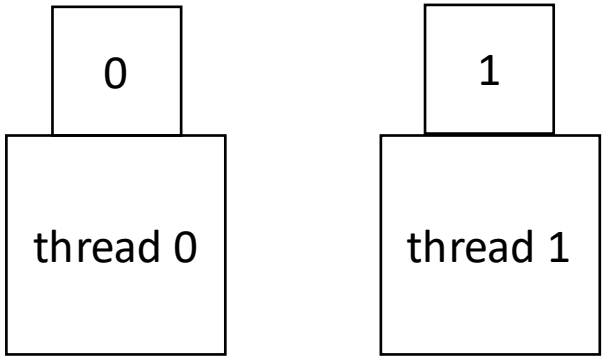
     // dynamic work based on x
  }
}
```

Work stealing - global worklist

- Global worklist: threads take tasks (iterations) dynamically




```
x: 2
0 - local_x - 0
1 - local_x - 1
```



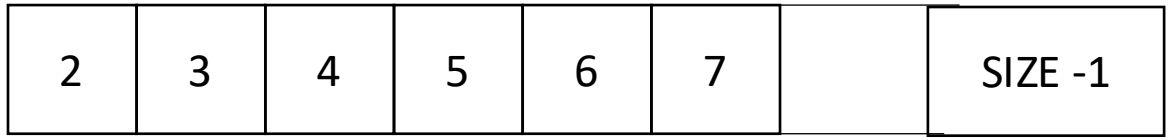

```
atomic_int x(0);
void parallel_loop(...) {

  for (int local_x = atomic_fetch_add(&x,1);
           local_x < SIZE;
           local_x = atomic_fetch_add(&x,1)) {

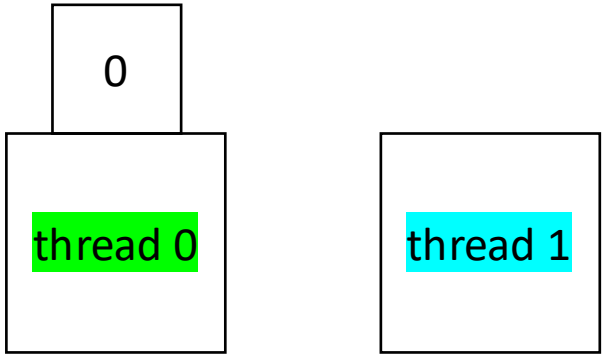
     // dynamic work based on x
  }
}
```

Work stealing - global worklist

- Global worklist: threads take tasks (iterations) dynamically




```
x: 2
0 - local_x - 0
1 - local_x - 1
```



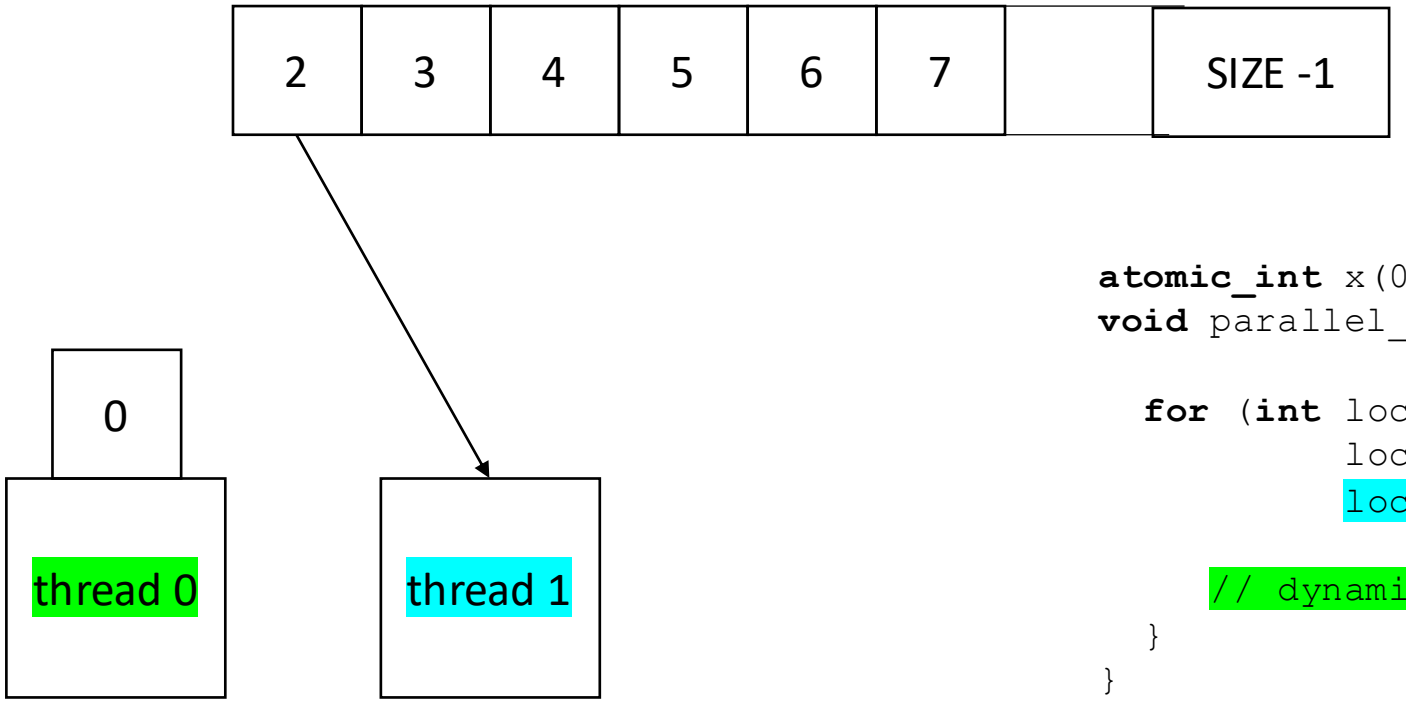

```
atomic_int x(0);
void parallel_loop(...) {

  for (int local_x = atomic_fetch_add(&x,1);
            local_x < SIZE;
            local_x = atomic_fetch_add(&x,1)) {

     // dynamic work based on x
  }
}
```

Work stealing - global worklist

- Global worklist: threads take tasks (iterations) dynamically




```
x: 3
0 - local_x - 0
1 - local_x - 2
```

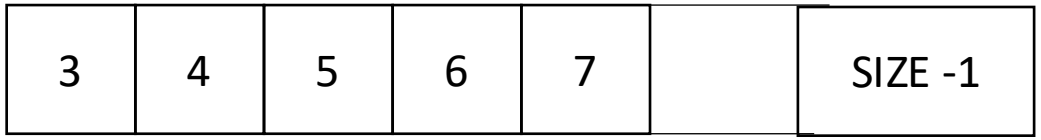
```
atomic_int x(0);
void parallel_loop(...) {

  for (int local_x = atomic_fetch_add(&x,1);
            local_x < SIZE;
            local_x = atomic_fetch_add(&x,1)) {

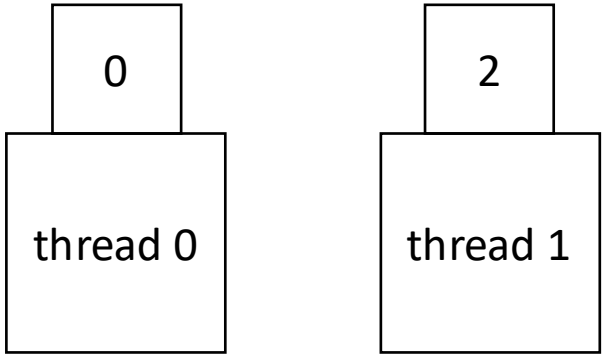
      // dynamic work based on x
  }
}
```

Work stealing - global worklist

- Global worklist: threads take tasks (iterations) dynamically




```
x: 3
0 - local_x - 0
1 - local_x - 2
```



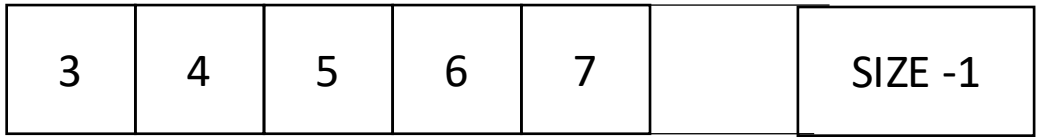

```
atomic_int x(0);
void parallel_loop(...) {

  for (int local_x = atomic_fetch_add(&x,1);
            local_x < SIZE;
            local_x = atomic_fetch_add(&x,1)) {

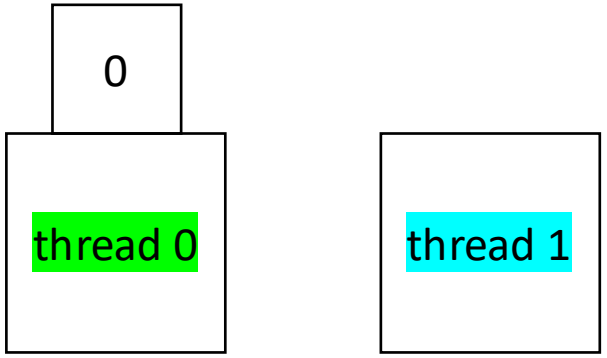
     // dynamic work based on x
  }
}
```

Work stealing - global worklist

- Global worklist: threads take tasks (iterations) dynamically




```
x: 3
0 - local_x - 0
1 - local_x - 2
```



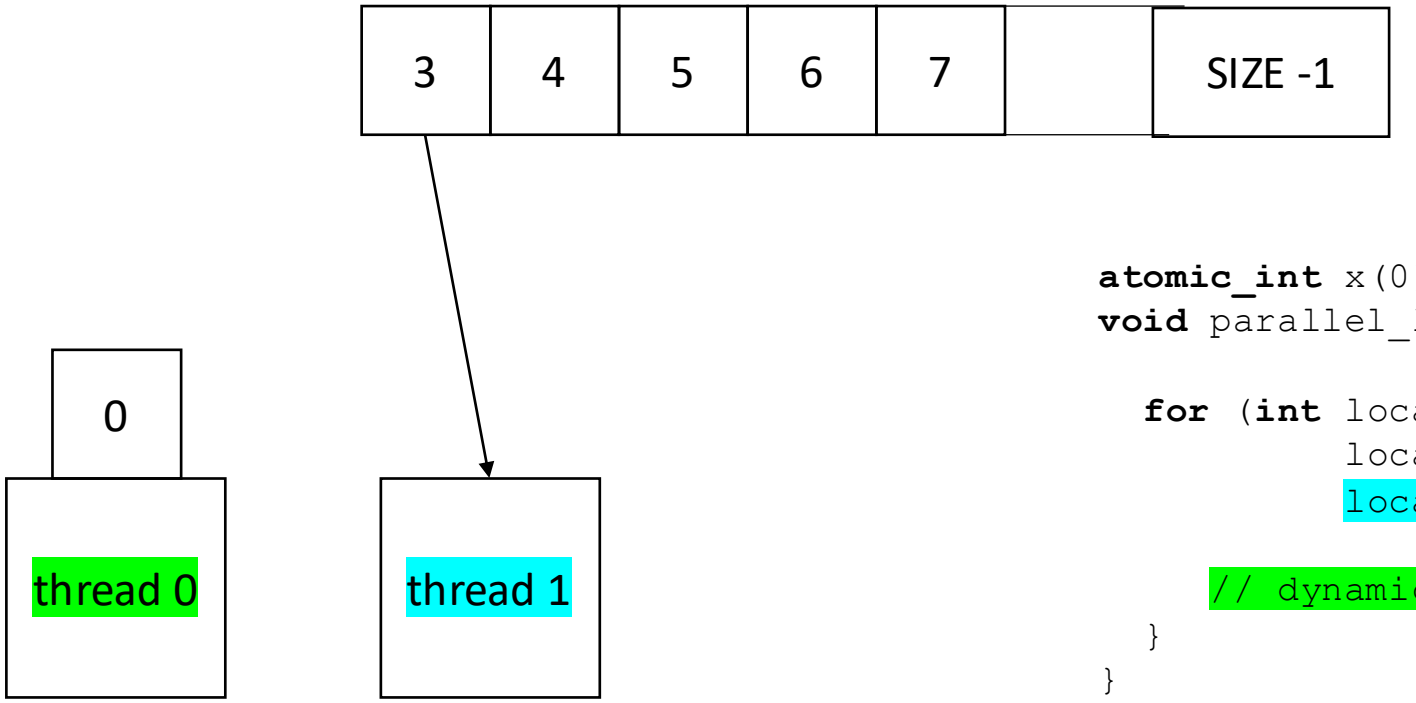

```
atomic_int x(0);
void parallel_loop(...) {

  for (int local_x = atomic_fetch_add(&x,1);
            local_x < SIZE;
            local_x = atomic_fetch_add(&x,1)) {

     // dynamic work based on x
  }
}
```

Work stealing - global worklist

- Global worklist: threads take tasks (iterations) dynamically




```
x: 4
0 - local_x - 0
1 - local_x - 3
```

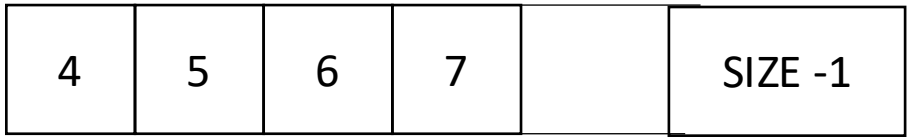
```
atomic_int x(0);
void parallel_loop(...) {

  for (int local_x = atomic_fetch_add(&x,1);
            local_x < SIZE;
            local_x = atomic_fetch_add(&x,1)) {

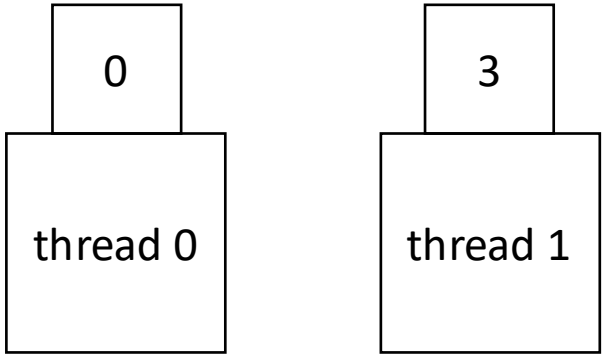
     // dynamic work based on x
  }
}
```

Work stealing - global worklist

- Global worklist: threads take tasks (iterations) dynamically




```
x: 4
0 - local_x - 0
1 - local_x - 3
```



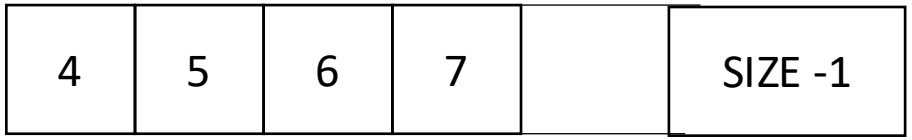

```
atomic_int x(0);
void parallel_loop(...) {

  for (int local_x = atomic_fetch_add(&x,1);
            local_x < SIZE;
            local_x = atomic_fetch_add(&x,1)) {

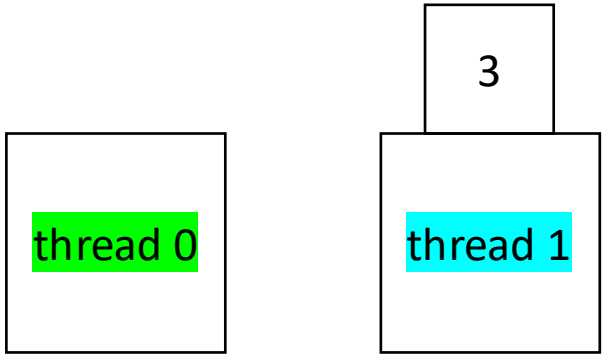
     // dynamic work based on x
  }
}
```

Work stealing - global worklist

- Global worklist: threads take tasks (iterations) dynamically




```
x: 4
0 - local_x - 0
1 - local_x - 3
```



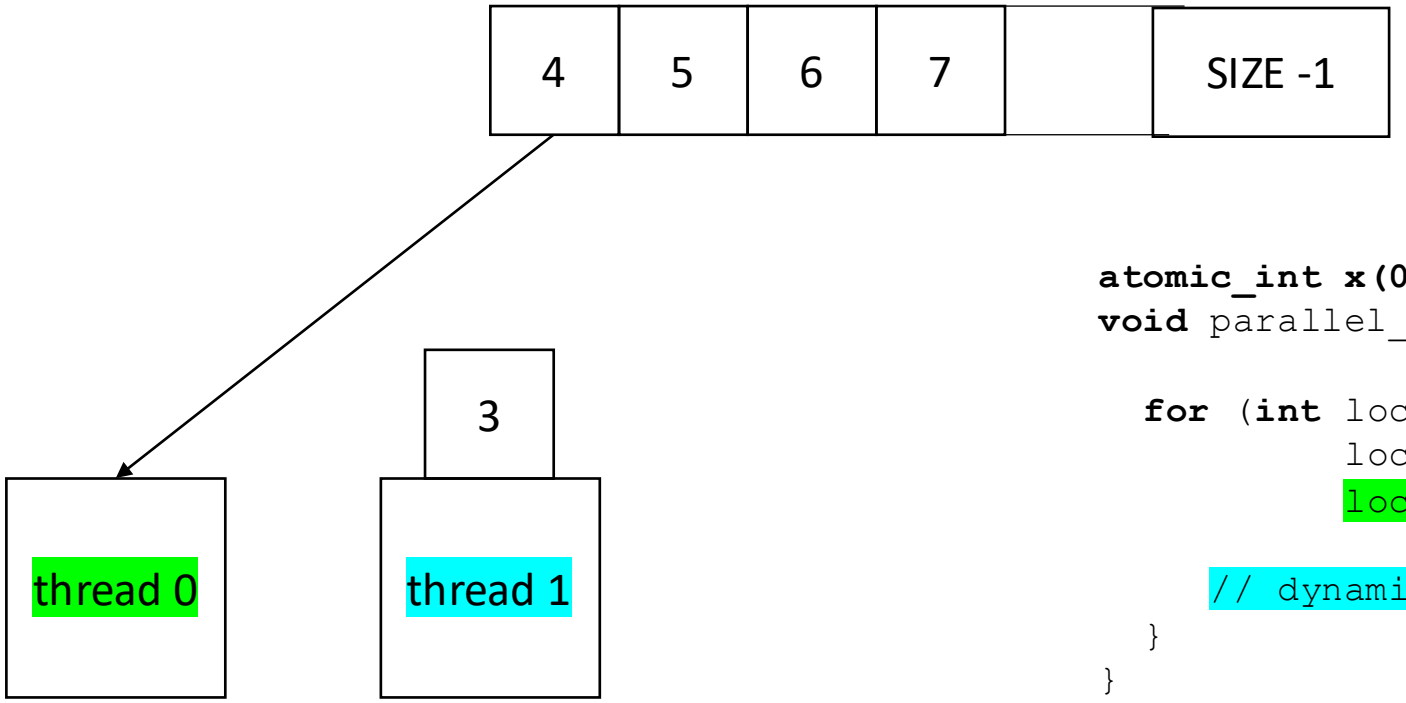

```
atomic_int x(0);
void parallel_loop(...) {

  for (int local_x = atomic_fetch_add(&x,1);
            local_x < SIZE;
            local_x = atomic_fetch_add(&x,1)) {

     // dynamic work based on x
  }
}
```

Work stealing - global worklist

- Global worklist: threads take tasks (iterations) dynamically




```
x: 5
0 - local_x - 4
1 - local_x - 3
```

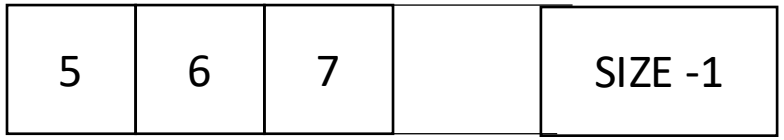
```
atomic_int x(0);
void parallel_loop(...) {

  for (int local_x = atomic_fetch_add(&x,1);
            local_x < SIZE;
            local_x = atomic_fetch_add(&x,1)) {

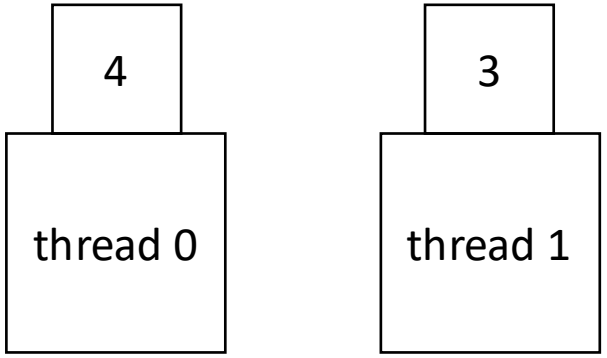
     // dynamic work based on x
  }
}
```

Work stealing - global worklist

- Global worklist: threads take tasks (iterations) dynamically




```
x: 5
0 - local_x - 4
1 - local_x - 3
```




```
atomic_int x(0);
void parallel_loop(...) {

  for (int local_x = atomic_fetch_add(&x,1);
            local_x < SIZE;
            local_x = atomic_fetch_add(&x,1)) {

     // dynamic work based on x
  }
}
```

Quiz 10.

Which one of the following is NOT a drawback of a global workstealing parallel schedule

Select one

A. Requires a concurrent data structure

B. Contention on shared cache lines

C. Contention on a single location with RMWs

Scan to answer

forms.gle/HrhkkgQiYqML2jcB9

chart: docs.google.com/forms/d/e/1FAIpQLSckAO0lpZ5Ng4HW5K7ZxxB4dclr8g6V9sTUHZoEJXz1CXRvHw/viewanalytics

Quiz 10.

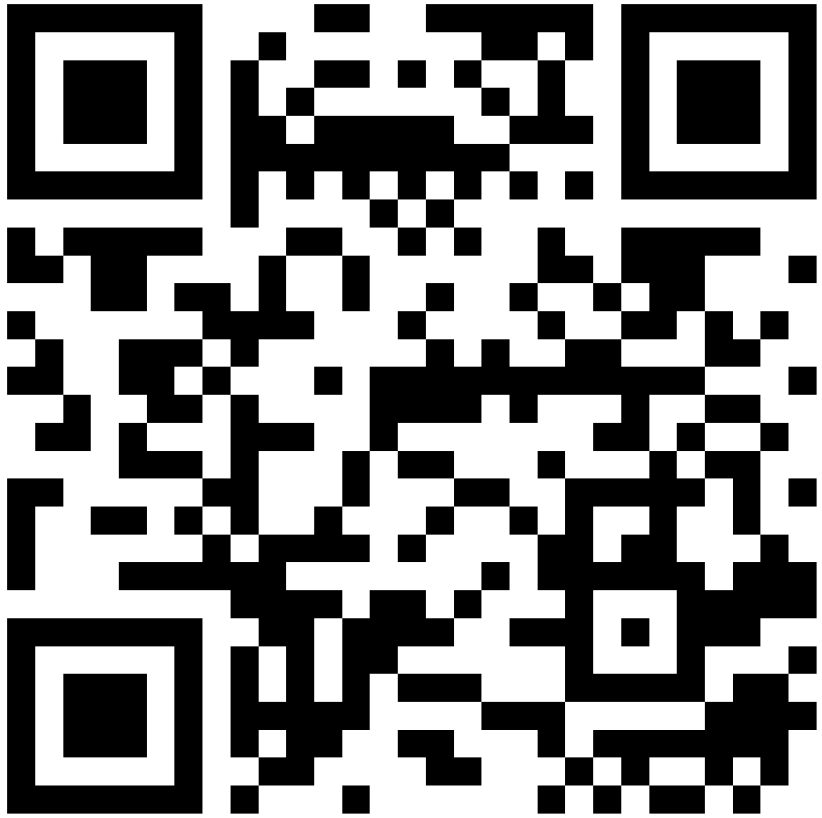
Which one of the following is NOT a drawback of a global workstealing parallel schedule

Select one

A. Requires a concurrent data structure

B. Contention on shared cache lines

C. Contention on a single location with RMWs



Scan to answer

A concurrent data structure is needed.

forms.gle/HrhkkgQiYqML2jcB9

chart: docs.google.com/forms/d/e/1FAIpQLSckAO0lpZ5Ng4HW5K7ZxxB4dclr8g6V9sTUHZoEJXz1CXRvHw/viewanalytics

Schedule

- DOALL Loops

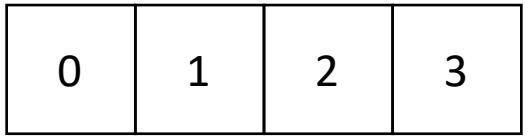
- **Parallel Schedules:**
  - Static
  - Global Worklists
  - **Local Worklists**

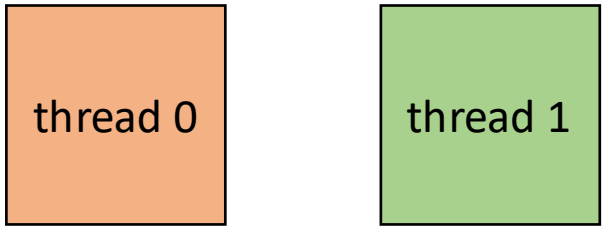
Work stealing - local worklists

- More difficult to implement
- low contention on local data-structures
- potentially better cache locality

Work stealing - local worklists

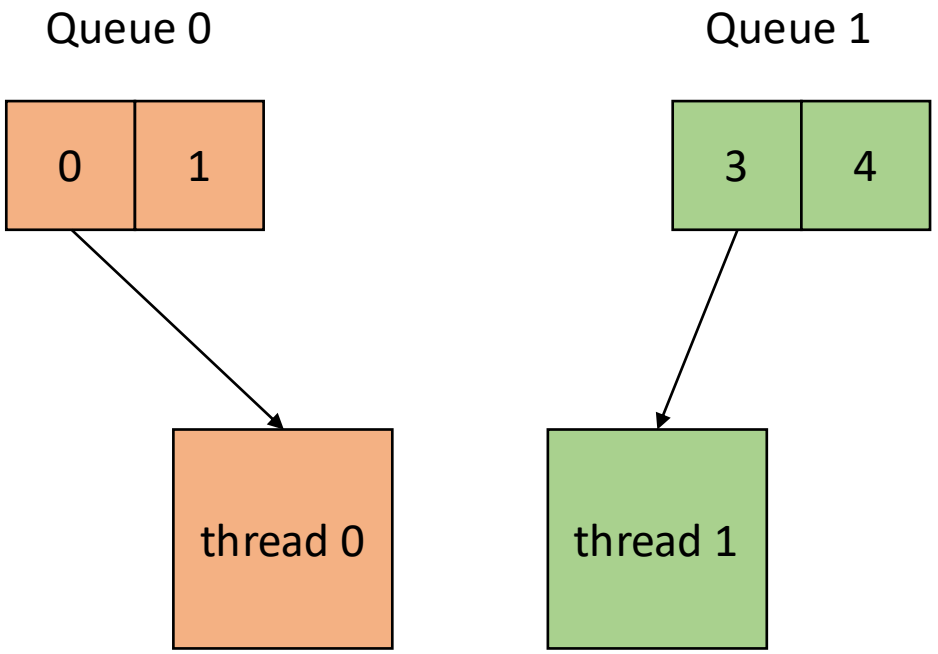
- local worklists: divide tasks into different worklists for each thread





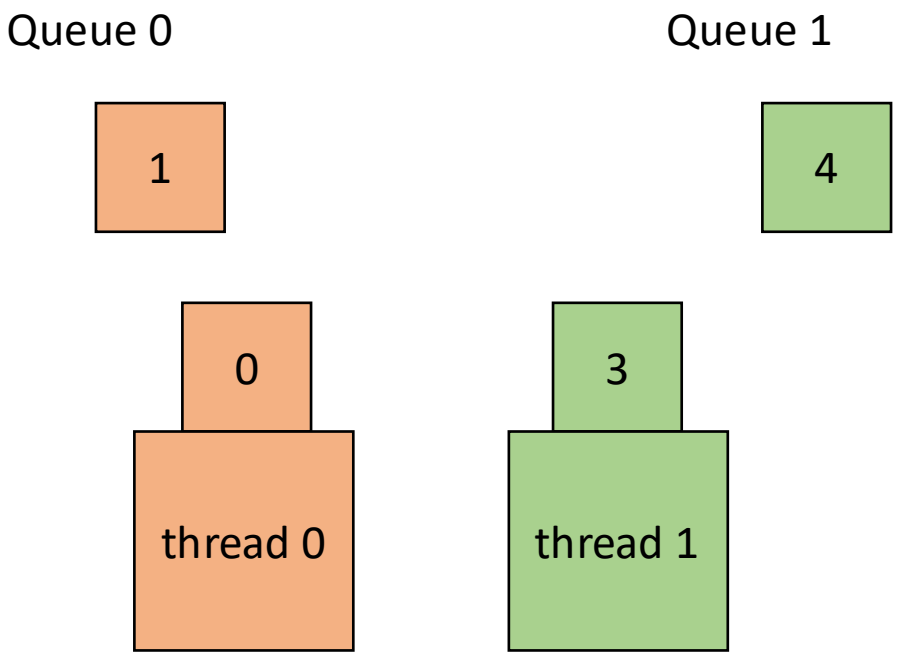
Work stealing - local worklists

- local worklists: divide tasks into different worklists for each thread



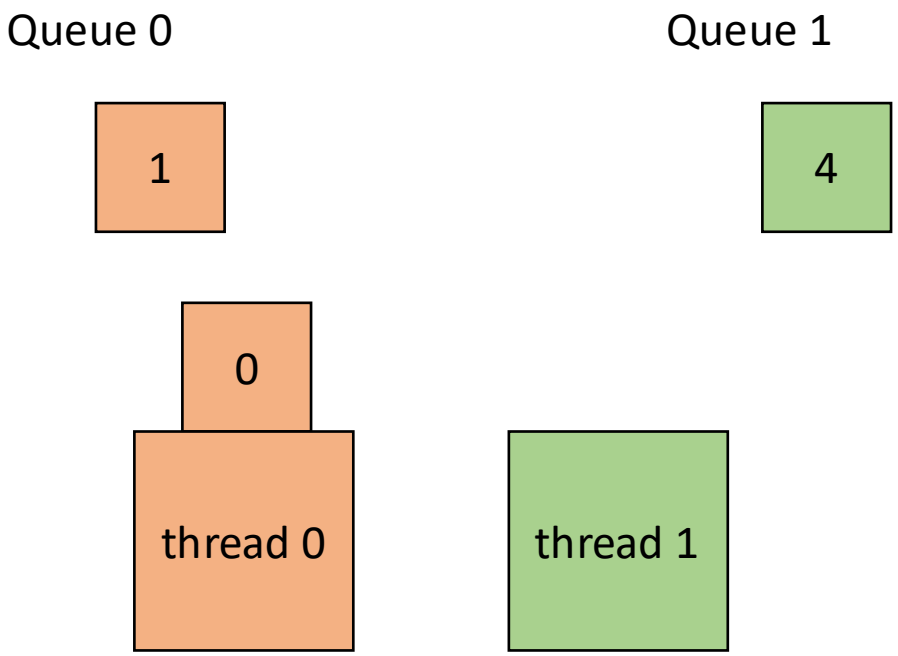
Work stealing - local worklists

- local worklists: divide tasks into different worklists for each thread



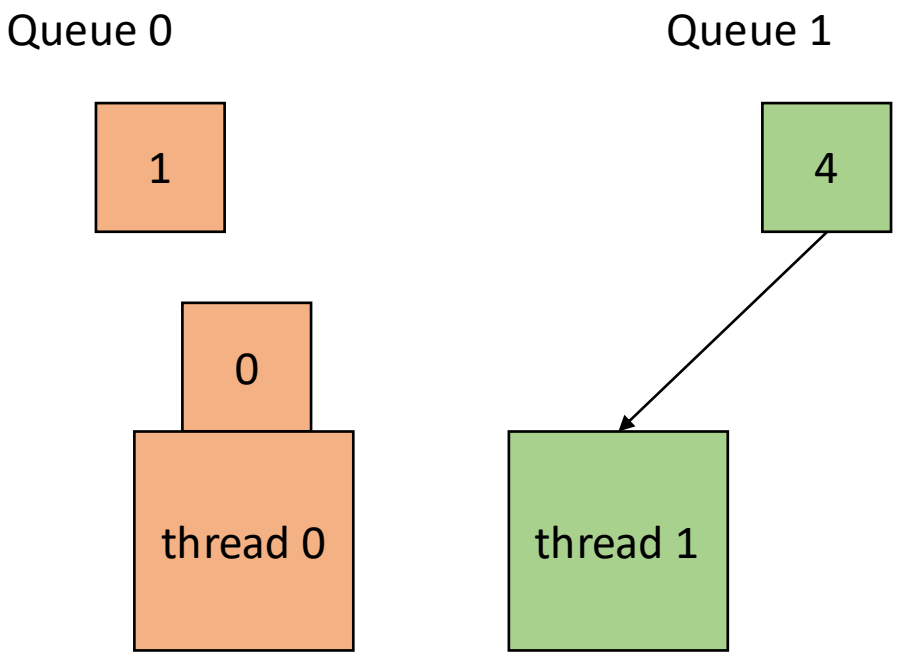
Work stealing - local worklists

- local worklists: divide tasks into different worklists for each thread



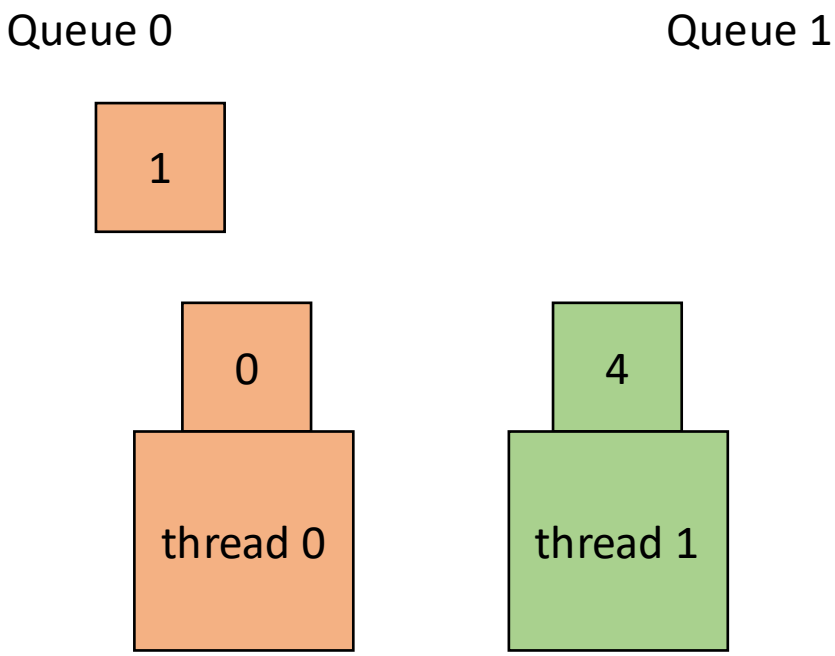
Work stealing - local worklists

- local worklists: divide tasks into different worklists for each thread



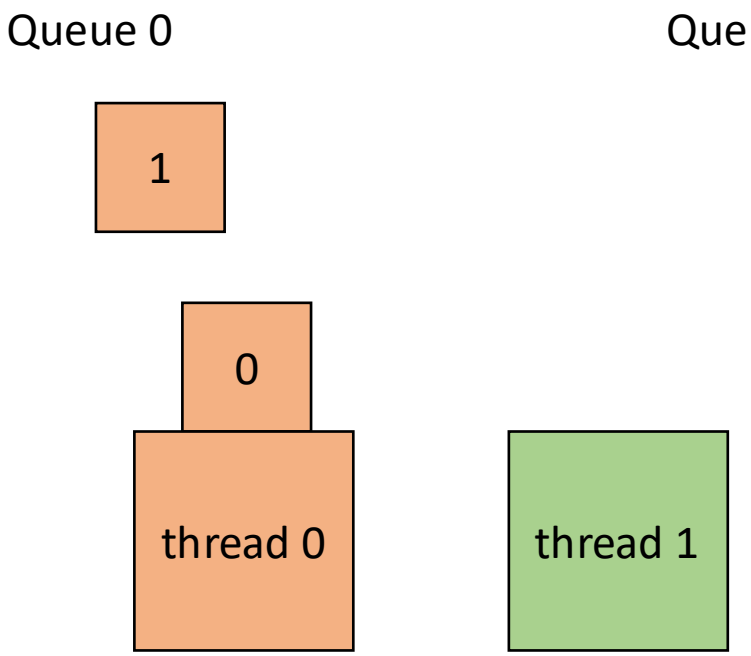
Work stealing - local worklists

- local worklists: divide tasks into different worklists for each thread



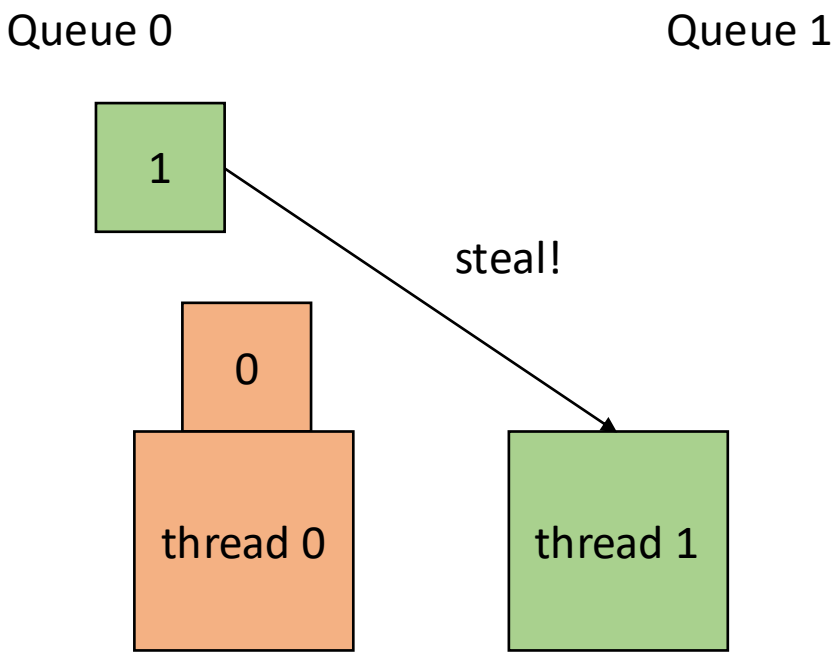
Work stealing - local worklists

- local worklists: divide tasks into different worklists for each thread



Work stealing - local worklists

- local worklists: divide tasks into different worklists for each thread

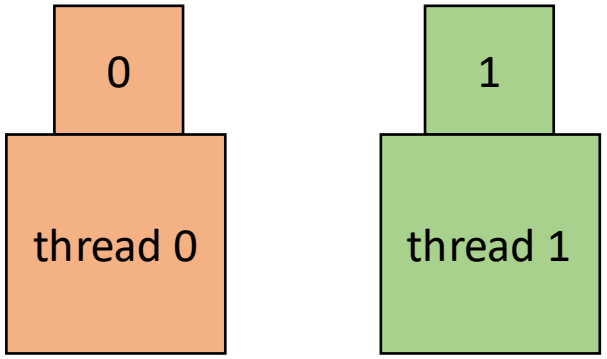


Work stealing - local worklists

- local worklists: divide tasks into different worklists for each thread

Queue 0

Queue 1



Work stealing - local worklists

- How to implement:

```
void foo() {
  ...
  for (x = 0; x < SIZE; x++) {
    // dynamic work based on x
  }
  ...
}
```

Work stealing - local worklists

- How to implement:

```
void foo() {
   ...
   for (x = 0; x < SIZE; x++) {
     // dynamic work based on x
   }
   ...
}
```

```
void parallel_loop(..., int tid) {

   for (x = 0; x < SIZE; x++) {
     // dynamic work based on x
   }
}
```

Make a new function, taking any variables used in loop body as args. Additionally take in a thread id

Work stealing - local worklists

- How to implement:

```
concurrent_queue cq[NUM_THREADS];
void foo() {
  ...
  for (x = 0; x < SIZE; x++) {
    // dynamic work based on x
  }
  ...
}
```

```
void parallel_loop(..., int tid) {

  for (x = 0; x < SIZE; x++) {
    // dynamic work based on x
  }
}
```

Make a global array of concurrent queues

Work stealing - local worklists

- How to implement:

```
concurrent_queue cq[NUM_THREADS];
void foo() {
  ...
  for (x = 0; x < SIZE; x++) {
    // dynamic work based on x
  }
  ...
}
```

```
void parallel_loop(..., int tid) {

  for (x = 0; x < SIZE; x++) {
    // dynamic work based on x
  }
}
```

What type of queues?

Make a global array of concurrent queues

Work stealing - local worklists

- How to implement:

```
concurrent_queue cq[NUM_THREADS];
void foo() {
  ...
  for (x = 0; x < SIZE; x++) {
    // dynamic work based on x
  }
  ...
}
```

```
void parallel_loop(..., int tid) {

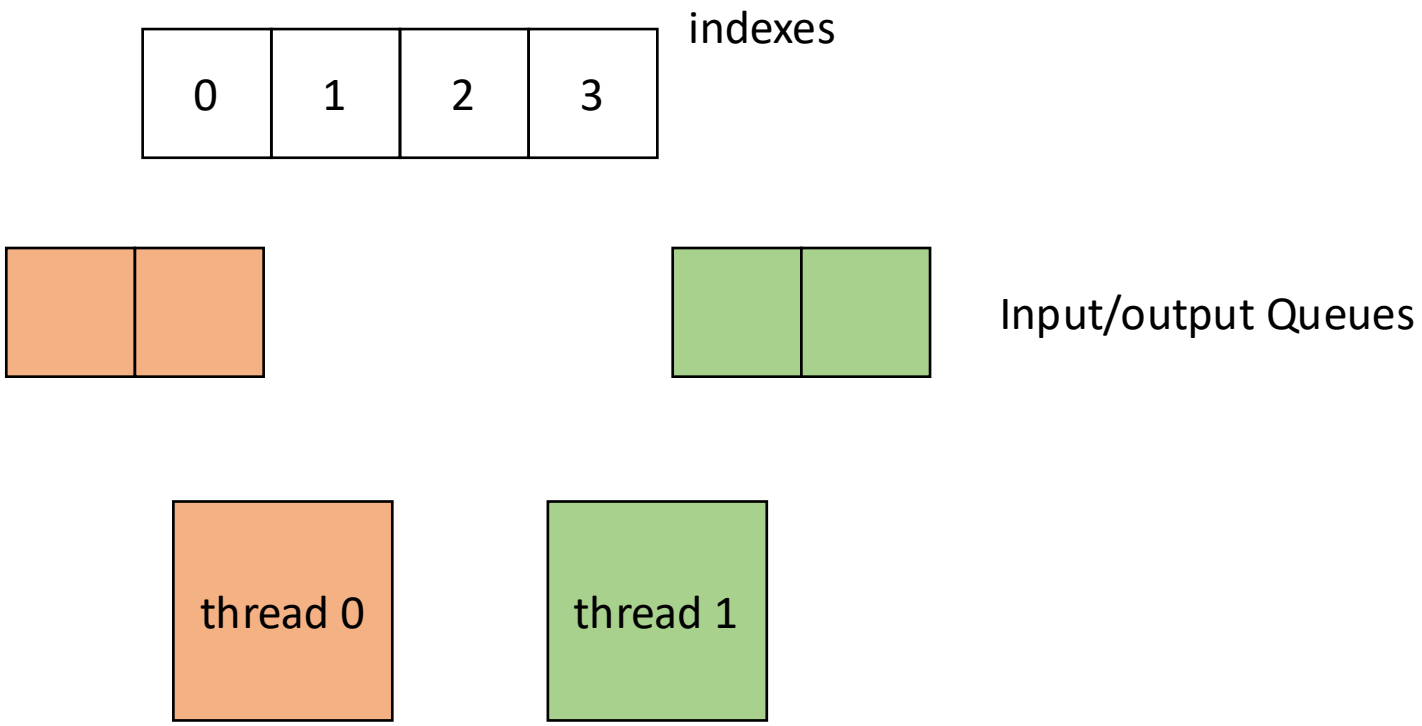
  for (x = 0; x < SIZE; x++) {
    // dynamic work based on x
  }
}
```

Make a global array of concurrent queues

What type of queues?
We're going to use InputOutput Queues!

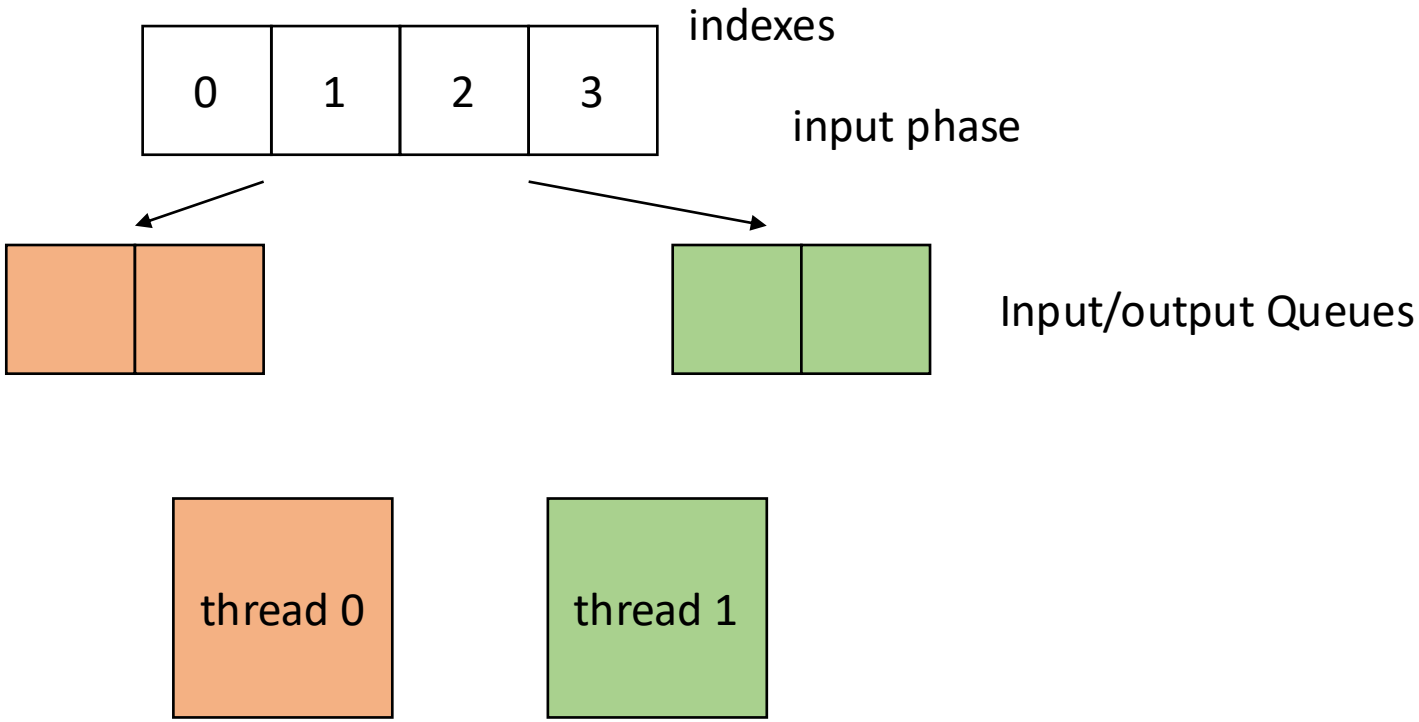
Work stealing - local worklists

- local worklists: divide tasks into different worklists for each thread



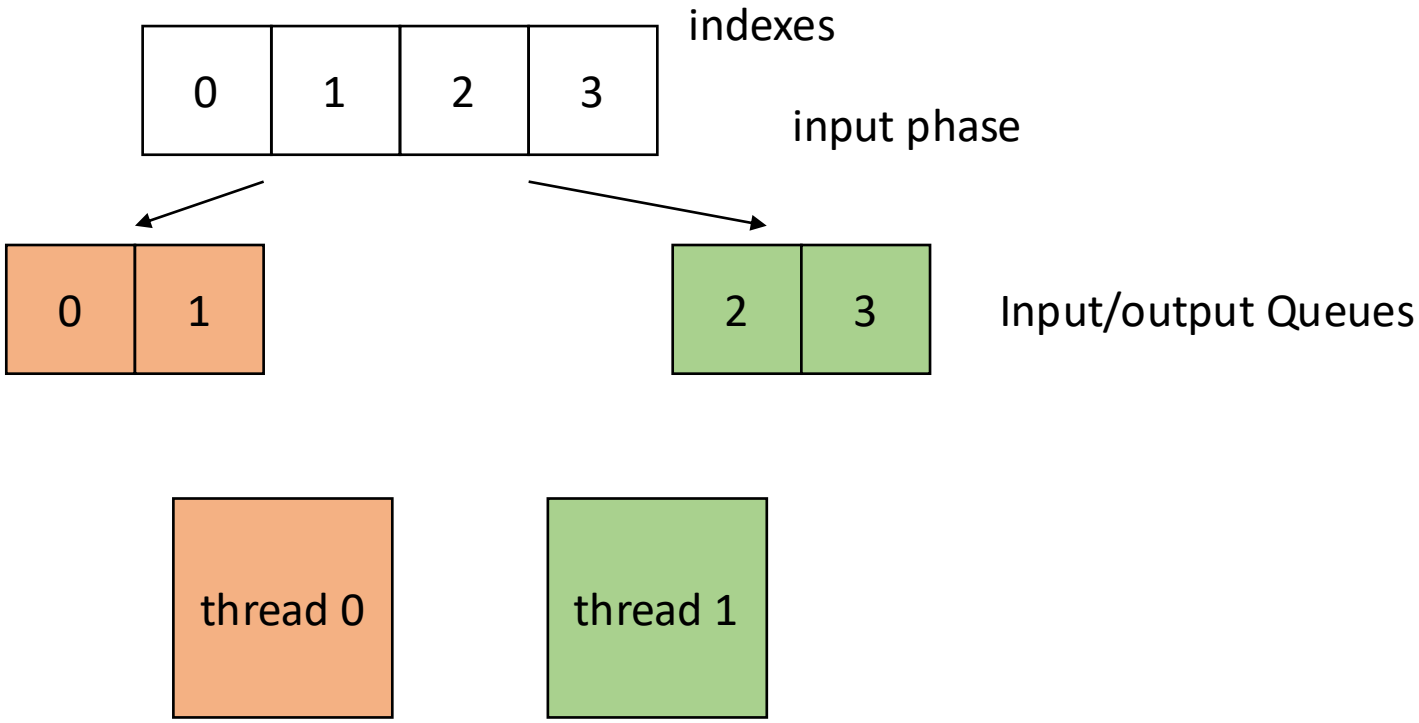
Work stealing - local worklists

- local worklists: divide tasks into different worklists for each thread



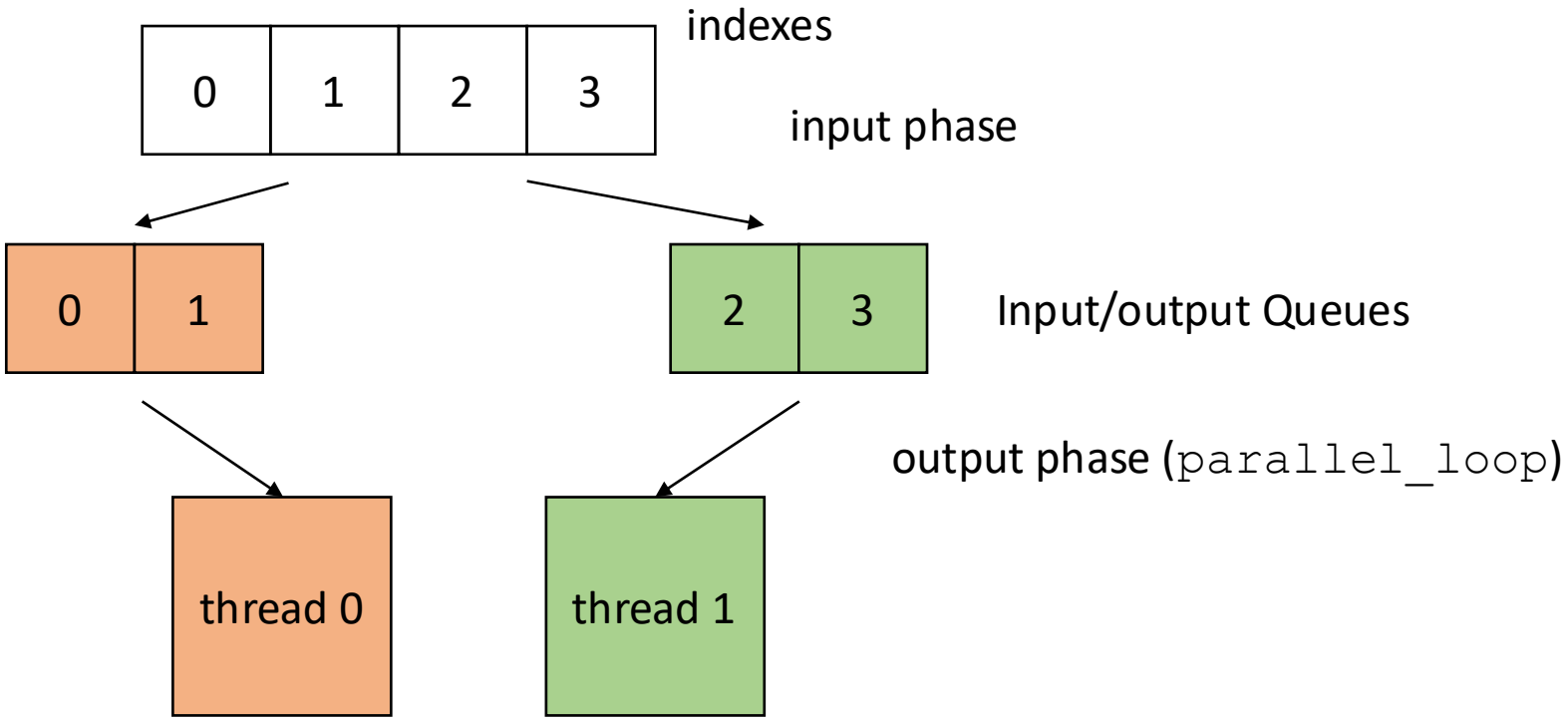
Work stealing - local worklists

- local worklists: divide tasks into different worklists for each thread



Work stealing - local worklists

- local worklists: divide tasks into different worklists for each thread



Work stealing - local worklists

```
concurrent_queues cq[NUM_THREADS];
void foo() {
   ...


   ...
}
```

First, we need to initialize the queues

Work stealing - local worklists

```
concurrent_queues cq[NUM_THREADS];
void foo() {
   ...
   // Spawn threads to initialize
   // join initializing threads

   ...
}
```

```
void parallel_enq(..., int tid, int num_threads)
{

  int chunk_size = SIZE / NUM_THREADS;
  int start = chunk_size * tid;
  int end = start + chunk_size;
  for (int x = start; x < end; x++) {
    cq[tid].enq(x);
  }
}
```

Just like the static schedule, except we are enqueuing

Work stealing - local worklists

```
concurrent_queues cq[NUM_THREADS];
void foo() {
   ...
   // Spawn threads to initialize
   // join initializing threads

   ...
}
```

Make sure to account for boundary conditions!

```
void parallel_enq(..., int tid, int num_threads)
{

  int chunk_size = SIZE / NUM_THREADS;
  int start = chunk_size * tid;
  int end = start + chunk_size;
  for (int x = start; x < end; x++) {
    cq[tid].enq(x);
  }
}
```

Just like the static schedule, except we are enqueuing

Work stealing - local worklists

- How to implement in a compiler:

```
NUM_THREADS = 2;
SIZE = 4;
CHUNK = 2;
```

x	0	1	2	3

tid	0	0	1	1

Make sure to account for boundary conditions!

```
void parallel_enq(..., int tid, int num_threads)
{

  int chunk_size = SIZE / NUM_THREADS;
  int start = chunk_size * tid;
  int end = start + chunk_size;
  for (int x = start; x < end; x++) {
    cq[tid].enq(x);
  }
}
```

Just like the static schedule, except we are enqueuing

Work stealing - local worklists

```
concurrent_queues cq[NUM_THREADS];
void foo() {
  ...
  // initialize queues
  // join threads

  // launch loop function
  ...
}
```

```
void parallel_loop(..., int tid, int num_threads) {

  for (x = 0; x < SIZE; x++) {
    // dynamic work based on x
  }
}
```

How do we modify the parallel loop?

Work stealing - local worklists

```
concurrent_queues cq[NUM_THREADS];
void foo() {
  ...
  // initialize queues
  // join threads

  // launch loop function
  ...
}
```

```
void parallel_loop(..., int tid, int num_threads) {

  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
}
```

loop until the queue is empty

Work stealing - local worklists

```
concurrent_queues cq[NUM_THREADS];
void foo() {
  ...
  // initialize queues
  // join threads

  // launch loop function
  ...
}
```

```
void parallel_loop(..., int tid, int num_threads) {

  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
}
```

loop until the queue is empty
Are we finished?

Work stealing - local worklists

```
concurrent_queues cq[NUM_THREADS];
void foo() {
   ...
   // initialize queues
   // join threads

   // launch loop function
   ...
}
```

```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

   int task = 0;
   for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
   {
      // dynamic work based on task
   }
   atomic_fetch_add(&finished_threads,1);
}
```

Track how many threads are finished

Work stealing - local worklists

```
concurrent_queues cq[NUM_THREADS];
void foo() {
   ...
   // initialize queues
   // join threads

   // launch loop function
   ...
}
```

```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

   int task = 0;
   for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
   {
     // dynamic work based on task
   }
   atomic_fetch_add(&finished_threads,1);
   while (finished_threads.load() != num_threads) {

   }
}
```

While there are threads that are still working

Work stealing - local worklists

```
concurrent_queues cq[NUM_THREADS];
void foo() {
   ...
   // initialize queues
   // join threads

   // launch loop function
   ...
}
```

```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

   int task = 0;
   for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
   {
     // dynamic work based on task
   }
   atomic_fetch_add(&finished_threads,1);
   while (finished_threads.load() != num_threads) {
     int target = // pick a thread to steal from
     int task = cq[target].deq();
   }
}
```

pick a random target and steal a task

Work stealing - local worklists

```
concurrent_queues cq[NUM_THREADS];
void foo() {
  ...
  // initialize queues
  // join threads

  // launch loop function
  // join loop threads
  ...
}
```

```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
      // perform task
    }
  }
}
```

Work stealing - local worklists

```
concurrent_queues cq[NUM_THREADS];
void foo() {
   ...
   // initialize queues
   // join threads

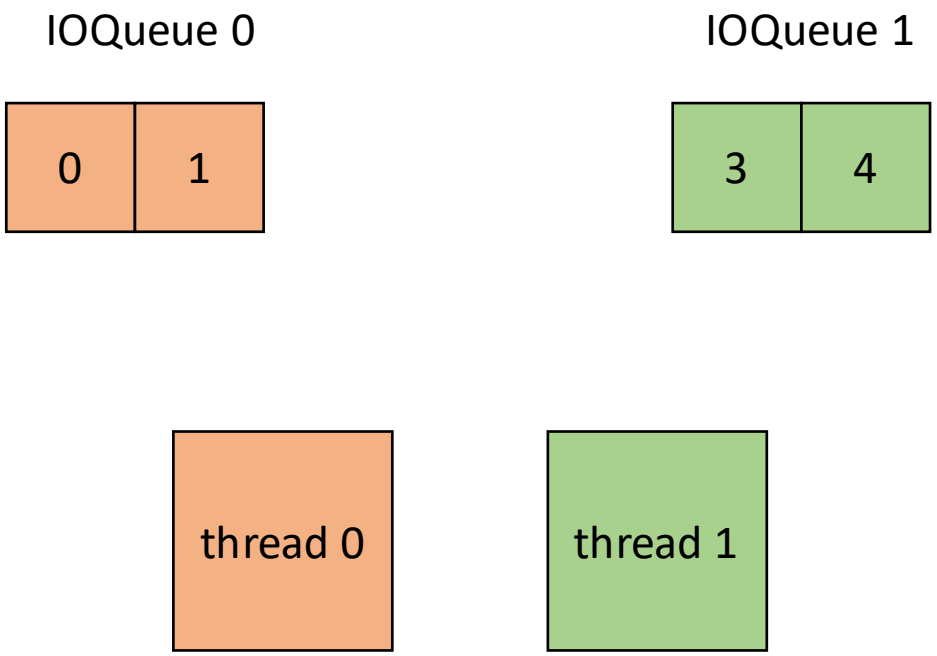
   // launch loop function
   // join loop threads
   ...
}
```

join the threads

```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

   int task = 0;
   for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
   {
      // dynamic work based on task
   }
   atomic_fetch_add(&finished_threads,1);
   while (finished_threads.load() != num_threads) {
      int target = // pick a thread to steal from
      int task = cq[target].deq();
      if (task != -1) {
         // perform task
      }
   }
}
```

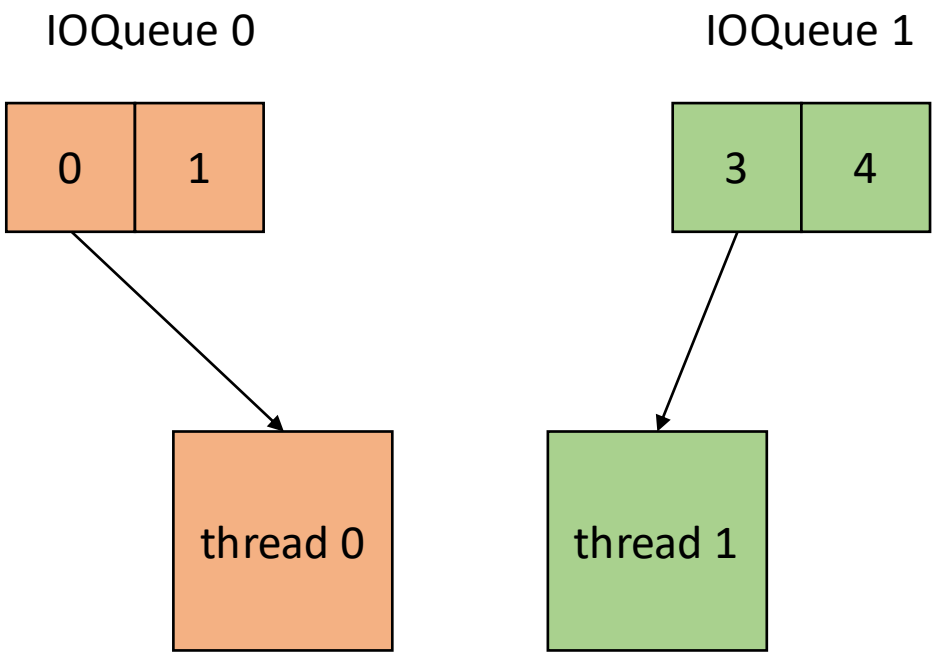
Work stealing - local worklists




```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
       // perform task
    }
  }
}
```

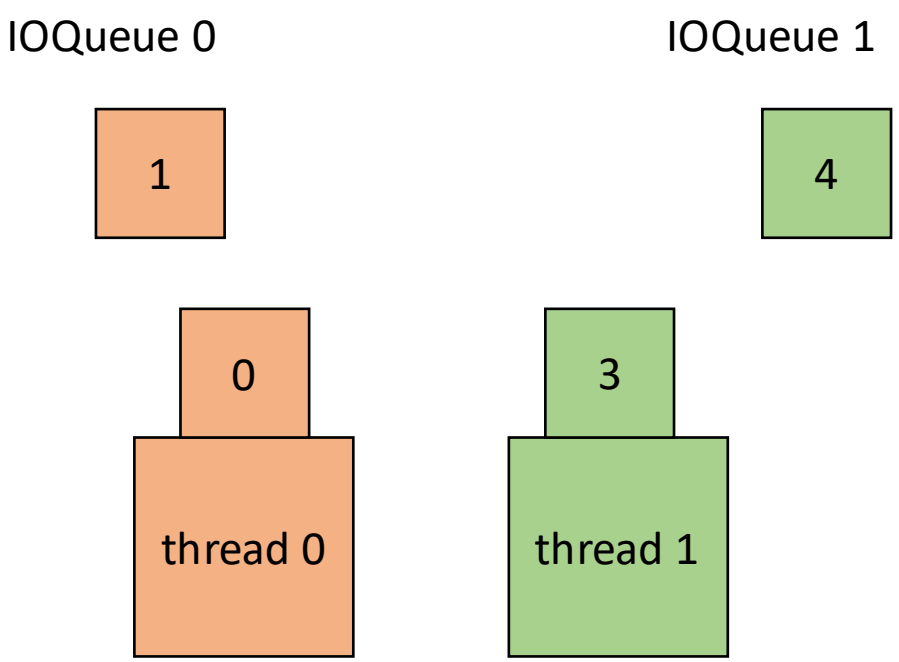
Work stealing - local worklists




```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
       // perform task
    }
  }
}
```

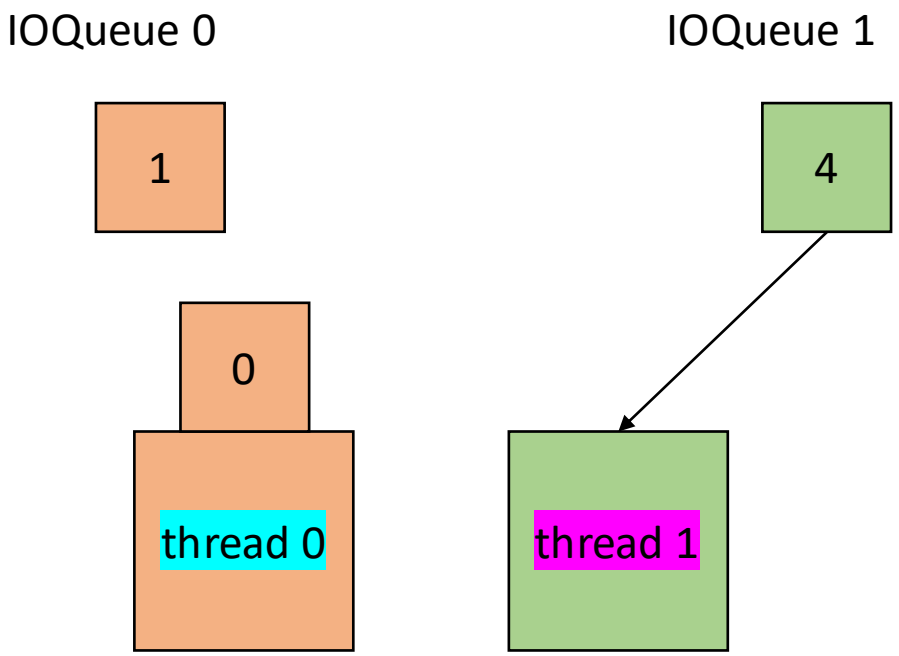
Work stealing - local worklists




```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
       // perform task
    }
  }
}
```

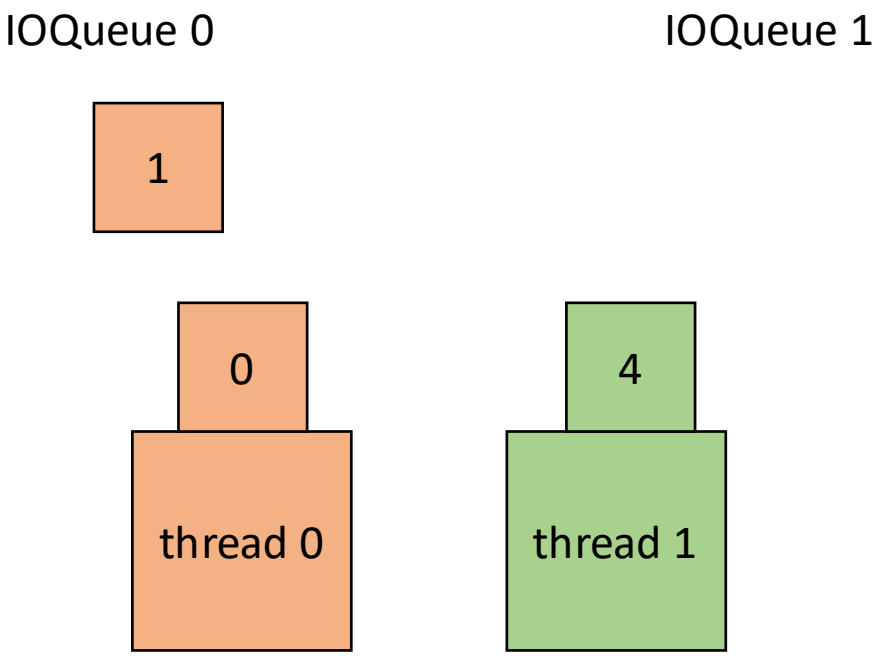
Work stealing - local worklists




```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
       // perform task
    }
  }
}
```

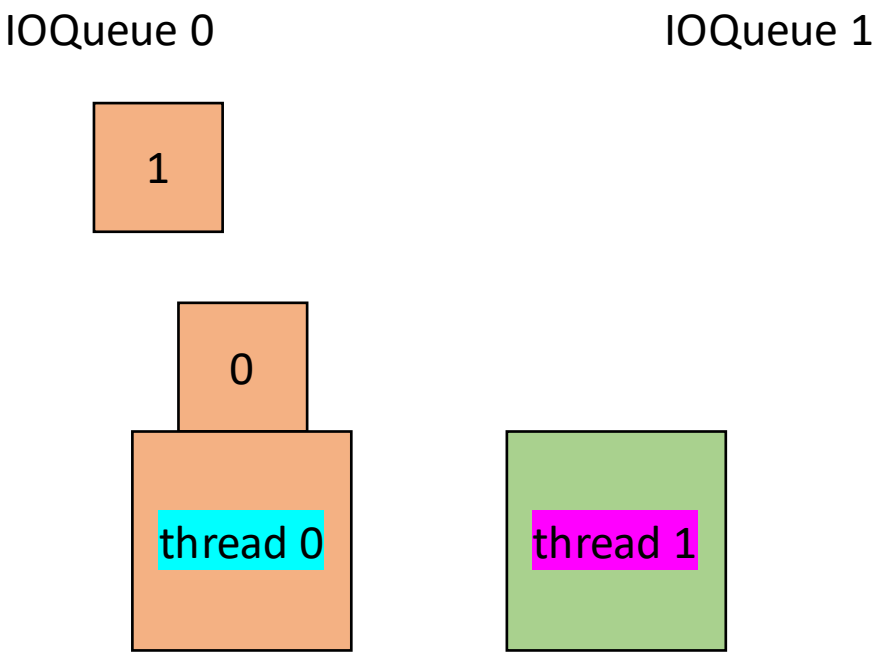
Work stealing - local worklists




```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
       // perform task
    }
  }
}
```

Work stealing - local worklists

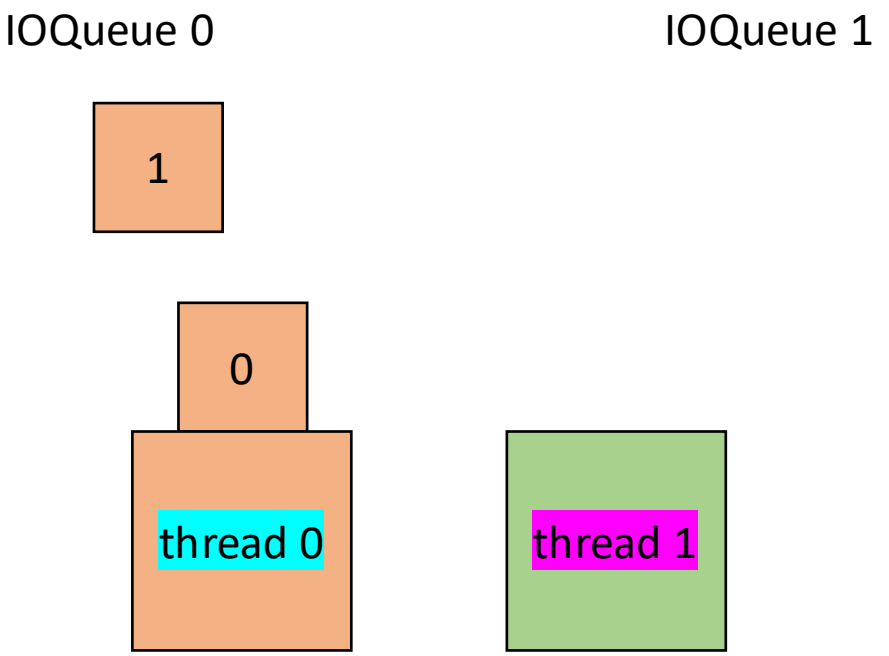



```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
       // perform task
    }
  }
}
```

Work stealing - local worklists

finished_threads: 1

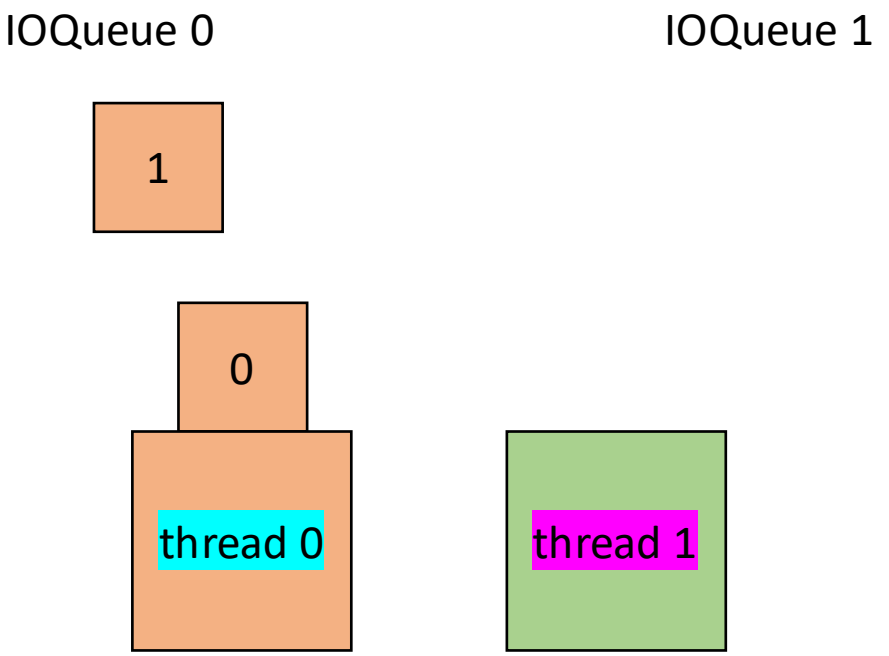



```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
       // perform task
    }
  }
}
```

Work stealing - local worklists

finished_threads: 1

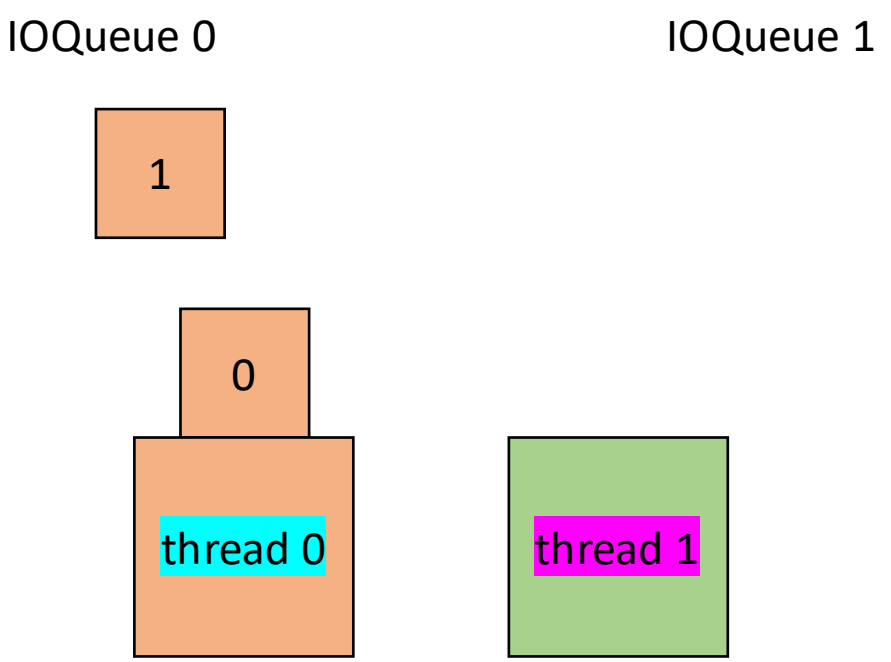



```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
       // perform task
    }
  }
}
```

Work stealing - local worklists

finished_threads: 1

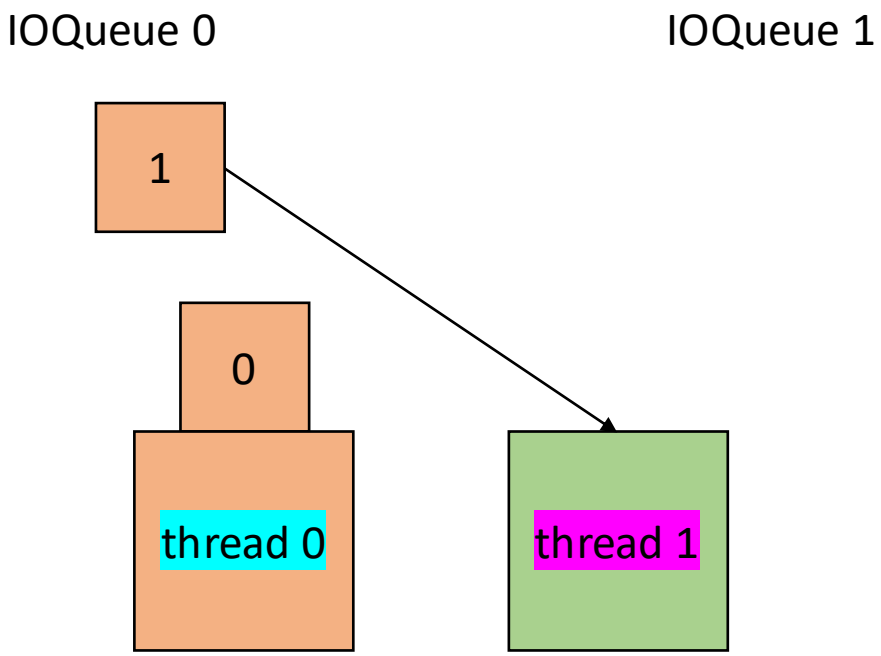



```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
       // perform task
    }
  }
}
```

Work stealing - local worklists

finished_threads: 1




```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

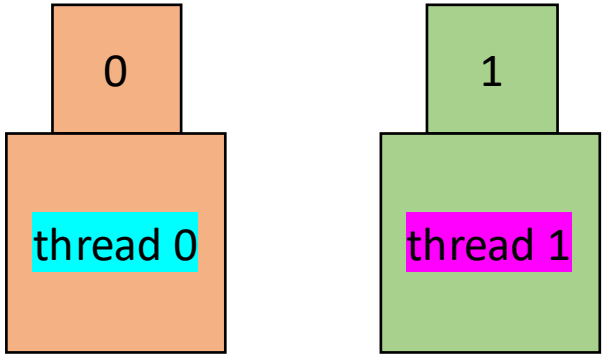
  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
       // perform task
    }
  }
}
```

Work stealing - local worklists

finished_threads: 1

IOQueue 0

IOQueue 1




```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

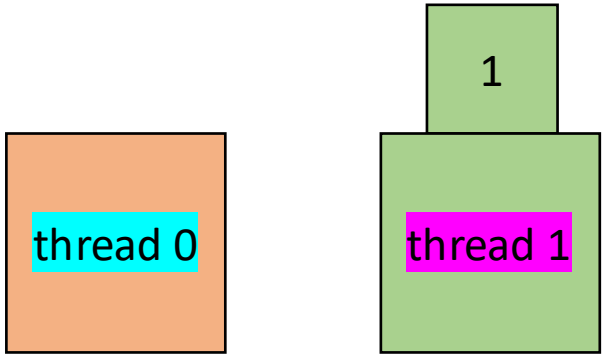
  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
       // perform task
    }
  }
}
```

Work stealing - local worklists

```
finished_threads: 1
```

IOQueue 0

IOQueue 1




```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

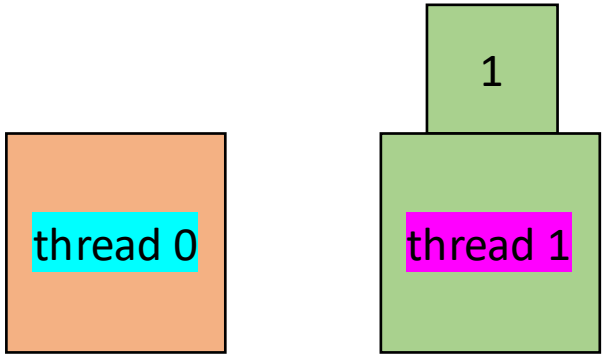
  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
       // perform task
    }
  }
}
```

Work stealing - local worklists

finished_threads: 2

IOQueue 0

IOQueue 1




```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
       // perform task
    }
  }
}
```

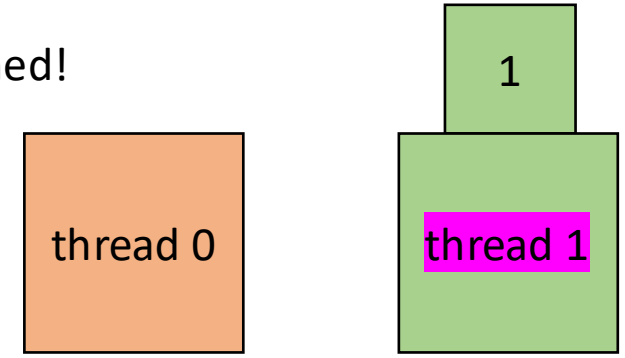
Work stealing - local worklists

finished_threads: 2

IOQueue 0

IOQueue 1

finished!




```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

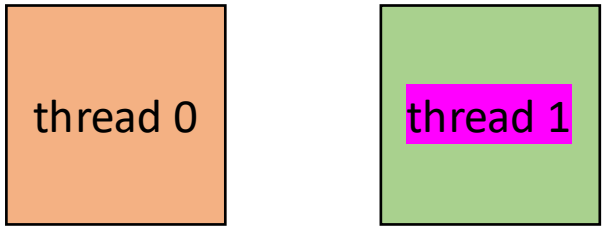
  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
       // perform task
    }
  }
}
```

Work stealing - local worklists

finished_threads: 2

IOQueue 0

IOQueue 1




```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

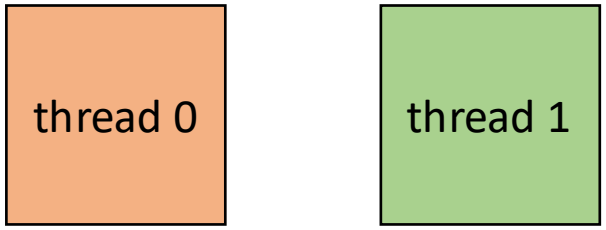
  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
       // perform task
    }
  }
}
```

Work stealing - local worklists

finished_threads: 2

IOQueue 0

IOQueue 1




```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
       // perform task
    }
  }
}
```

Quiz 10.

Which of the following is NOT an overhead of the local worklist workstealing parallel schedule.

Select one

A. Initialization of the queues

B. Checking a global variable to ensure all work is completed

C. Managing concurrent enqueues to the worklists



Scan to answer

forms.gle/MmDQEFBLaqSTjkCm8

chart: docs.google.com/forms/d/e/1FAIpQLSe6ULaZ2AbL0nqH2QodJl9cJWas3N6XlaAJ4JPpO7jzyawL5A/viewanalytics

Quiz 10.

Which of the following is NOT an overhead of the local worklist workstealing parallel schedule.

Select one

A. Initialization of the queues

B. Checking a global variable to ensure all work is completed

C. Managing concurrent enqueues to the worklists

There were no concurrent enqueues.

Scan to answer

forms.gle/MmDQEFBLaqSTjkCm8

chart: docs.google.com/forms/d/e/1FAIpQLSe6ULaZ2AbL0nqH2QodJl9cJWas3N6XlaAJ4JPpO7jzyawL5A/viewanalytics

Quiz 10.

Write a few sentences about the pros and cons of using local workstealing queues over the global worklist.

Free response: a few sentences on the form

Scan to answer

forms.gle/VdDu2GrUrNz7noef9

chart: docs.google.com/forms/d/e/1FAIpQLSf7H1ZVBLzzNO-8XiHyBSIcF56WEH8PW5BMuFJ-d8tDuC9fNw/viewanalytics

Quiz 10.

Write a few sentences about the pros and cons of using local workstealing queues over the global worklist.

Free response: a few sentences on the form

Simpler vs. More efficient.

Scan to answer

forms.gle/VdDu2GrUrNz7noef9

chart: docs.google.com/forms/d/e/1FAIpQLSf7H1ZVBLzzNO-8XiHyBSIcF56WEH8PW5BMuFJ-d8tDuC9fNw/viewanalytics

Open mp

- Code examples